mathematics

MDPI

Review

Beyond Neural Solvers: A Critical Review of Machine Learning for Combinatorial Optimization

Mostafa E. A. Ibrahim, Alaa E. S. Ahmed * and Yassine Daadaa

College of Computer and Information Sciences, Imam Mohammad Ibn Saud Islamic University (IMSIU), Riyadh 11432, Saudi Arabia; meibrahim@imamu.edu.sa (M.E.A.I.); ymdaadaa@imamu.edu.sa (Y.D.)
* Correspondence: asmohamed@imamu.edu.sa

Abstract

Combinatorial optimization is a key component in critical decision problems such as routing, scheduling, network design, and graph optimization. Although combinatorial optimization methods, including exact algorithms, approximation methods, constraint programming, mixed integer programming, and metaheuristics, are widely available, they often face obstacles, such as limited scalability and adaptability in various applications. In this study, a systematic critical review of machine learning for combinatorial optimization is provided to characterize the usage and evaluation of learning-based approaches. A detailed analysis is used to infer and determine findings and limitations. The paper emphasizes how machine learning for computational optimization has changed over time, moving from end-to-end neural solvers to hybrid systems. Learning components are essential for directing, speeding up, or enhancing traditional solver backbones such as constraint programming and metaheuristics in hybrid systems. The review also critically examines current limits that impact performance in general, including scalability, deployment readiness, generalization, and benchmark consistency. Even though using large language models for problem formulation and heuristic synthesis has potential, more work needs to be done to ensure reliable validation. As a conclusion, this article examines recent studies' findings, emphasizes the growing trend toward hybrid learning-driven optimization frameworks, and underlines important methodological limits and unresolved issues.

Keywords: combinatorial optimization; machine learning; neural combinatorial optimization; reinforcement learning; graph neural networks; hybrid neuro-symbolic optimization; learning-augmented metaheuristics; large language models; robust and explainable optimization

MSC: 90C27

Check for updates

Academic Editor: Jonathan Blackledge

Received: 7 May 2026
Revised: 6 June 2026
Accepted: 16 June 2026
Published: 19 June 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Combinatorial optimization (CO) lies at the core of operations research (OR), computer science, artificial intelligence (AI), and engineering decision-making. A wide range of real-world decision problems, such as vehicle routing, Max-Cut, Set Cover, resource allocation, the knapsack problem (KP), production scheduling, the Maximum Satisfiability problem (MAX-SAT), job-shop scheduling, network design, facility location, and computation offloading, involve finding the optimal choice within an exponentially large solution space. Many common versions of these problems are known to be NP-hard, and they are typically described as discrete, limited, and computationally costly. Therefore, exact methods, constraint programming (CP), approximating algorithms, hybrid IP, local search, and

metaheuristics have historically been used to address CO. Even though these traditional approaches are still very important, their efficacy often depends on domain-specific heuristics, handcrafted modeling assumptions, solver configuration, and substantial computational resources, particularly in large, dynamic, noisy, or real-time environments.

In the last decade, machine learning (ML), deep learning (DL), reinforcement learning (RL), graph neural networks (GNNs), decision-focused learning, and, more recently, large language models (LLMs) have been widely considered as tools for solving or assisting in CO problems. Instead of relying exclusively on handcrafted rules, learning-based methods use data-driven approaches to infer useful structures from data, predict candidate solutions, direct search, initialize solvers, choose neighborhoods, improve branching, generate heuristics, or approximate difficult optimization components.

Recent research has demonstrated this fast methodological advancement, including neural CO, ML-enhanced metaheuristics, RL-based optimization, GNN-based solvers, predict-then-optimize pipelines, quantum-inspired formulations, and LLM-assisted heuristic synthesis [1–6]. This shift from handcrafted optimization to learning-based solver components and hybrid neuro-symbolic systems is already recognized in the reviewed publications as a key advancement in contemporary CO research. Despite this advance, the field remains fragmented. Different benchmark sets, instance generators, solver baselines, metrics, hardware settings, and reporting standards are frequently used by various research communities to assess learning-based CO techniques. For example, reward-based performance, makespan, latency, or solution quality are commonly used to assess RL-based approaches. Approximation ratios, initial gaps, or feasibility rates are frequently reported by GNN-based techniques, whereas later stages of regret or choice loss are the focus of prediction-and-optimization techniques. Energy consumption, productivity, revenue, equity, and resource use are examples of application-dependent metrics used in field research. Predict-then-optimize approaches focus on regret or downstream decision loss. GNN-based approaches frequently report approximation ratios, primal gaps, or feasibility rates. Domain-specific studies employ application-dependent metrics like energy consumption, throughput, revenue, fairness, or resource utilization. Consequently, due to this variability, it is not possible to clearly address or identify the exact cause of the observed improvement. The improvement may result from several reasons, such as a limited experimental environment, genuine methodological progress, the choice of a suitable benchmark, or a weak baseline. However, there is some evidence of generalization shown in many experiments. The evidence is extracted based on the instance sizes, distributions, problem variants, or real-world deployment conditions in these experiments.

Another challenge faced by the learning-based optimization approaches is the tendency to prefer experimental performance to traditional guarantees. Pure neural solver methods usually produce fast and high-quality solutions, but at the same time, their optimality assurance or feasibility certificates are not guaranteed. Also, they lead to bad behavior under distribution shifts. RL-based solvers offer a limited dependency on labelled optimal solutions, but at the same time, they show inefficient performance in terms of samples and sensitivity to reward shaping. Graphical neural networks provide strong relational inductive biases; however, their continuous output must still be transformed to fulfill discrete restrictions through the process of projection, decoding, or repairing. Predict-then-optimize approaches often call for costly differentiability approximations and an optimization oracle. However, they can match the learning goals with subsequent decisions. LLM-based optimization can be used in different applications, such as natural language modeling and code generation. These approaches face challenges related to latency, cost, benchmark contamination, reproducibility, and the lack of formal guarantees [7,8].

As a result, this study takes a systematic critical review approach. It provides a mapping of the main learning paradigms utilized in combinatorial optimization. It assesses their advantages and disadvantages and contrasts their methodological functions. Additionally, it identifies unmet research needs. It emphasizes the growing convergence toward hybrid neuro-symbolic optimization. These frameworks include learning models in "exact solvers, constraint programming, metaheuristics," search procedures, and mathematical programming pipelines rather than taking the place of conventional optimization techniques.

1.1. Motivation

This review is motivated by four key factors. First, despite the quick development of ML-for-CO, a common critical synthesis contrasting learning models across problem categories and solver architectures is still lacking. Existing studies tend to focus on individual techniques without adequately addressing how these paradigms relate to or converge with one another. Consequently, there is difficulty among researchers in differentiating between robust methodologies and those that are still in the early stages of development. Second, comparing ML-for-CO across papers is a challenging issue due to the variability of the quality of the baselines, the instability of benchmark standardization, and insufficient statistical validation. In CO, the performance is highly affected by solver time limits, instance structure, parameter tuning, and hardware. Therefore, in this critical review, both the performance and the quality of the approaches are determined. Third, there is always a gap between benchmark performance and real-world reliability. Real-world applications such as healthcare scheduling and cloud and edge resource management necessitate explainability, reproducibility, and scalability. Fourth, employing LLMs in CO problems moves the focus to some methodological issues that do not exist with traditional ML models, such as data contamination, closed-model reproducibility, inference cost, and the verification of generated solutions.

1.2. Key Contributions

This article makes the following contributions:

1. It introduces a mathematical formulation of the formal basis for understanding learning-based approaches to combinatorial optimization.
2. It develops a taxonomy of machine learning techniques for combinatorial optimization.
3. It critically evaluates ML-for-CO approaches from a variety of angles, such as feasibility, heuristic selection, solution generation, and solver support, to identify methodological limitations.
4. It demonstrates the move in the field from purely end-to-end neural solvers toward hybrid neuro-symbolic optimization that incorporates learned components with symbolic optimization backbones.
5. It highlights LLMs and multimodal foundation models as the latest advances in ML-for-CO.
6. It determines open research issues that are essential to the field's advancement.

1.3. Paper Outline

The rest of this article is structured as follows. The review protocol, which includes the search strategy, inclusion and exclusion criteria, and data extraction procedure, is presented in Section 2. The technical basis of combinatorial optimization and learning-based solvers, including formal CO formulations, traditional optimization techniques, and the function of machine learning in optimization pipelines, is presented in Section 3. Major methodological paradigms, including reinforcement learning, graph neural networks, ML-enhanced metaheuristics, predict-then-optimize frameworks, quantum-inspired optimization, LLM-based

techniques, and domain-specific applications, are critically synthesized in Section 4. The main methodological trends, limits, research gaps, the increasing convergence toward hybrid neuro-symbolic optimization, and the future research directions are outlined in Section 5. Finally, the study is concluded in Section 6.

2. Review Protocol

In order to prepare this review article in a consistent and reproducible manner, a systematic process that adheres to the PRISMA 2020 guidelines for reviewing the literature was applied. The systematic process involves subsequent stages as follows: early at the beginning, a clear review area definition and, consequently, the appropriate main research questions were settled. Secondly, the authors searched highly reputable academic databases for relevant publications. Thirdly, the authors refined the search results by quick title and abstract assessment. Fourthly, the authors defined a clear inclusion and exclusion criteria to determine the appropriateness of full-manuscript retrieval. Finally, the retrieved articles were thoroughly explored to identify their methodology, significance, empirical validity, scaling, reproducibility, and shortcomings.

2.1. Research Concern and Questions

Despite rapid growth in ML, DL, and conceptual modeling approaches for CO, this research field is still scattered into various methodological themes such as neural combinatorial optimization, RL, GNNs, learning-augmented metaheuristics, predict-then-optimize pipelines, quantum-inspired methods, and emerging LLM-based heuristic synthesis. There is little critical synthesis of how different approaches vary in terms of scaling, generality, feasibility assurances, robustness, reproducibility, and practical implementation, despite the fact that existing studies frequently claim encouraging results on individual benchmarks. Thus, the systematic mapping and critical evaluation of the development, methodological contributions, constraints, and future research prospects of machine learning-based CO techniques is the focus of this review.

The following fundamental research question serves as the basis for this review: What are the methodological contributions, limitations, and unresolved issues of ML, DL, and LLM-based approaches to combinatorial optimization? To answer this research question, the authors identify and investigate the following sub-research questions. (i) How can we categorize current relevant research efforts? (ii) Which CO problem classes and application-specific domains were prevalently deliberated in the literature? (iii) Which learning models and solver architectures prevailed in the literature? (iv) How did ML-assisted approaches for CO compare to conventional metaheuristic optimization benchmarks? (v) To what extent do these approaches extend their methodologies across different problem sizes and distributions? (vi) What are the challenging issues regarding scaling, reliability, reproducibility, clarity of interpretation, and deployment aspects?

2.2. Search Methodology and Selection Criteria

We searched for reliable electronic resources, such as Web of Science (WoS), IEEE Xplore, SpringerLink, ScienceDirect/Elsevier, Wiley Online Library, MDPI, and top-indexed conference proceedings, in order to compile this review paper. Publications from 2020 to 2026 were included in the search, which concentrated on research on large language model techniques for combinatorial optimization, deep learning, and machine learning. Learning-based optimization, neural combinatorial optimization, RL for combinatorial problems, graph neural network solvers, learning-augmented metaheuristics, predict-then-optimize techniques, and foundation model-assisted optimization are among the research topics covered by the database search terms.

In order to look at publication patterns pertaining to the subject of this review, the authors searched the Web of Science (WoS) database. More than 1516 research publications were found when the search term "Machine Learning for Combinatorial Optimization" was first used. A total of 1147 publications were found once the search was narrowed to only include studies released between 2020 and 2026. Every search was carried out on 23 March 2026. A yearly count of articles published throughout the chosen time is displayed in Figure 1, which depicts the recently published trend in this field of study and shows an ongoing rise in academic curiosity. The distribution of the resulting searching records by document type, namely, articles, review articles, conference proceeding papers, and book chapters, is illustrated in Figure 2a. Lastly, Figure 2b shows how the search results were distributed among publishers, showing the publication venues that supported machine learning research for combinatorial optimization between 2020 and 2026.

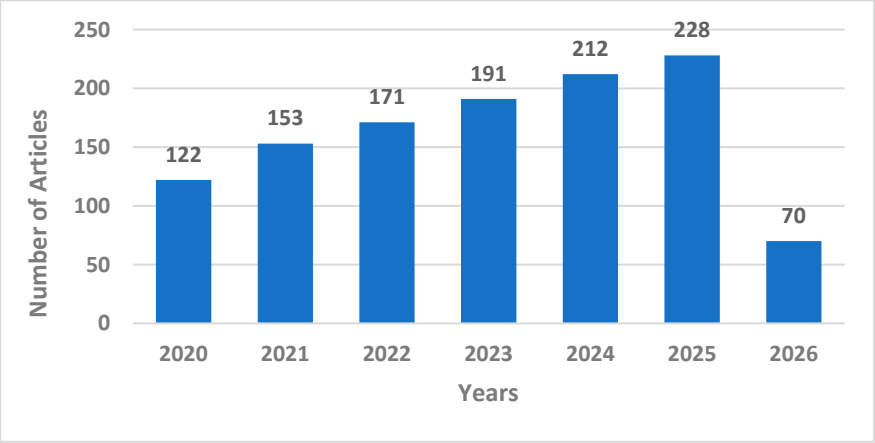
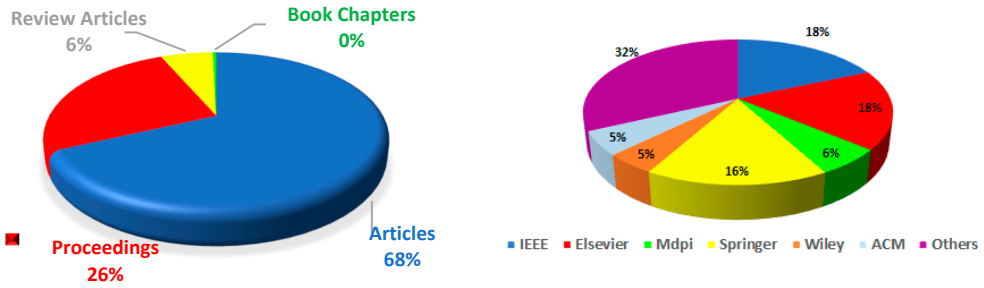


Figure 1. A yearly count of articles published between 2020 and 2026 from WoS.



(**a**) (**b**)

Figure 2. Informatics of retrieved WoS search records (**a**) by document type and (**b**) by publisher.

Boolean operators and restricted keyword combinations were used in the database search to increase reproducibility and transparency. The primary search term was ("machine learning" OR "deep learning" OR "reinforcement learning" OR "graph neural network" OR "large language model" OR "foundation model" OR "neural solver" OR "learning-augmented") AND ("combinatorial optimization" OR "combinatorial optimization" OR "integer programming" OR "mixed-integer programming" OR "constraint programming" OR "vehicle routing" OR "travelling salesman problem" OR "scheduling" OR "knapsack" OR "Max-Cut" OR "Set Cover" OR "MAX-SAT"). To increase coverage, more method-dependent keywords were employed, such as ("predict-then-optimize" OR "decision-focused learning" OR "differentiable optimization"), ("learning-enhanced metaheuristics" OR "hyper-heuristics" OR "large neighborhood search"), ("QUBO" OR "Ising" OR "quantum-inspired optimization"), and ("LLM" OR "large language model"

OR "heuristic generation" OR "code generation") AND ("combinatorial optimization" OR "vehicle routing" OR "scheduling"). While maintaining the same conceptual structure, queries were modified to fit the searching syntax of each database.

There were two steps in the screening process. In order to eliminate obviously unrelated records, duplications, non-English publications, preprints, abstract-only conference items, and articles published outside of the 2020–2026 timespan, two reviewers separately assessed titles and abstracts in the first stage. Records were kept for full-text evaluation if their appropriateness could not be identified with certainty from the title and abstract. Full texts were assessed using the inclusion and exclusion criteria in the second phase. Discussions were used to settle disagreements among reviewers, and studies were only included when agreement was established.

The studies that were kept following full-text screening were subjected to a systematic quality assessment mechanism. The following criteria were used to evaluate each article: (i) significance for ML-based or learning-assisted combinatorial optimization; (ii) clearness of the CO problem definition; (iii) adequateness of the methodology explanation; (iv) proneness of datasets, benchmarks, or instance-generation standards; (v) robustness and importance of baseline comparisons; (vi) clearness of assessment measures; (vii) proof of scaling or generalization assessment; (viii) reproducibility metrics, such as code, data, solver settings, or implementation details; and (ix) explicit discussion of limitations. Research studies that either lacked adequate scientific or conceptual conclusions, had unclear methodology, were irrelevant to CO, or had subpar reporting were omitted in addition to the previously stated exclusion criteria. To ensure that the final included list of references supported a methodical critical synthesis, a quality evaluation approach was put in place. As shown in Figure 3, the study identification and screening stages are presented.

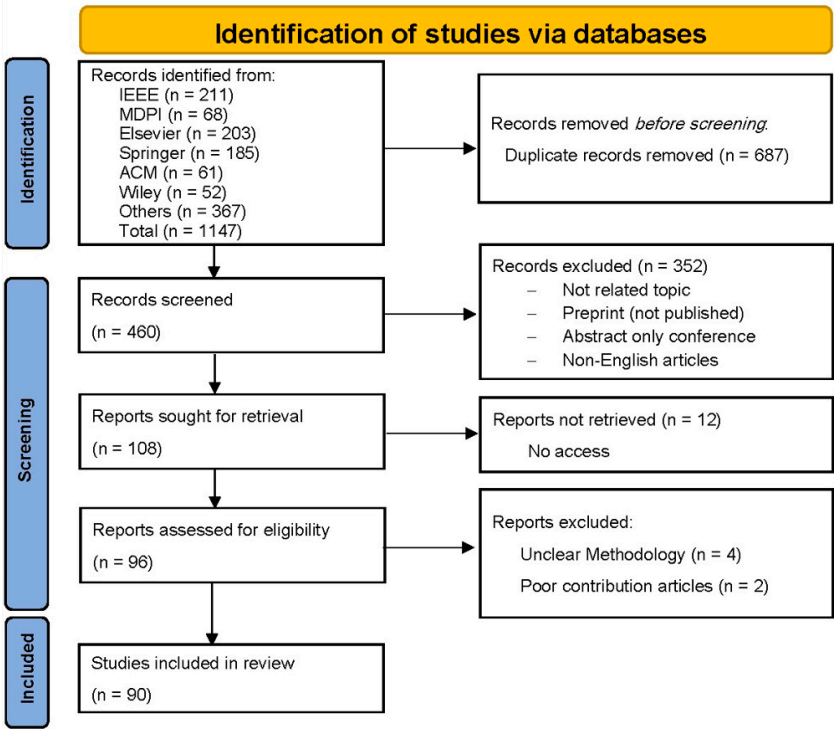


Figure 3. PRISMA 2020 flow diagram for study identification, screening, eligibility assessment, and inclusion.

To ensure consistency and comparability across the examined literature, data from the selected research articles was collected in the second phase using a standard extraction form. The form includes information about publication information, goals of research,

nature of CO problem, application domain, learning method, model architecture, optimization strategy, benchmarking data, evaluation indicators, baseline techniques, reported performance, and significant drawbacks. Additionally, it noted whether the suggested methods included foundation model components, graph-based learning, RL, constraint programming, mixed IP, metaheuristics, or classical solvers. A methodical and critical synthesis of methodological trends, empirical findings, and research needs in machine learning-based CO was made possible by this structured procedure.

3. Technical Background: Combinatorial Optimization and Learning-Based Solvers

In this section, the mathematical formulation of the formal basis for understanding learning-based approaches to combinatorial optimization is introduced. Combinatorial optimization is the process of selecting an optimal discrete solution out of a set of defined feasible CO configurations.

It establishes the underlying mathematical basis for many problems, such as routing, scheduling, assignment, packing, resource allocation, graph optimization, and selection, which are encountered in different areas such as operations research, computer science, engineering, manufacturing, transportation, energy systems, and communication networks [1,2,9]. A generic CO problem can be expressed as

$$x^* = \arg\min_{x \in X} \mathrm{f}(x; I), \tag{1}$$

where $\mathrm{f}(x; I)$ represents the objective function associated with problem instance I, x is a discrete decision vector, and x^* denotes the feasible solution set. This formulation encompasses a wide class of problems, including the traveling salesman problem (TSP), vehicle routing problem (VRP), knapsack problem, Max-Cut, Set Cover, MAX-SAT, job-shop scheduling, machine scheduling, and resource allocation problems [4,6,10]. Many of these problems are NP-hard, implying that exact solution methods may be computationally expensive and require exponential time in the worst case unless P = NP. This inherent computational complexity has driven CO toward the use of exact algorithms, approximation methods, problem-specific heuristics, and metaheuristics [9,10]. Recently, for this reason, machine learning is increasingly being explored as a tool for accelerating or guiding optimization. In a standard binary or mixed-integer linear programming formulation, the problem can be mathematically expressed as follows:

$$\begin{aligned} &\min_{x} c^T x \\ &s.t\ A_x \leq b \\ &x_j \in \{0,1\},\ j \in B, \\ &x_j \in \mathbb{Z}, \quad j \in \mathbb{Z}_{set} \end{aligned} \tag{2}$$

where c denotes the cost vector, A and b represent the constraint system, B indexes binary variables, and $\mathbb{Z}_{set}$ indexes general integer variables. This formulation is considered a core base to classical optimization due to its common structure for assignment, scheduling, routing, covering, packing, and network design problems [9]. It is also important for learning-based CO, as many methods do not fully replace mathematical programming but instead assist it by estimating high-quality primal solutions or warm-starting solvers, choosing branching variables, predicting the importance of variables, learning limitations, or search steering [11,12]. In recent ML-based solutions to CO problems, the hybrid perspectives have been adopted, especially when integrating learned components with others such as mixed-integer programming, constraint programming, large-neighborhood search, or metaheuristics [11,13].

A typical example of a canonical CO problem is the TSP. In this problem, a set of cities $V = \{1, ..., n\}$ and pairwise travel costs d_{ij} are given to find a minimum-cost Hamiltonian tour. A binary edge selection variable x_{ij} is used, where its value is equal to 1 if the tour travels directly from city i to j, which is expressed by the following compact formulation:

$$\begin{aligned} &\min_{x} \sum_{i \in V} \sum_{j \in V,\, j \neq i} d_{ij}\, x_{ij} \\ &s.t \sum_{j \in V,\, j \neq i} x_{ij} = 1, \quad \forall i \in V, \\ &\quad \sum_{i \in V,\, i \neq j} x_{ij} = 1, \quad \forall j \in V, \\ &\quad \sum_{i \in S} \sum_{j \in S,\, j \neq i} x_{ij} \leq |S| - 1, \quad \forall S \subset V,\, 2 \leq |S| \leq N - 1, \\ &\quad x_{ij} \in \{1, 0\}. \end{aligned} \tag{3}$$

The first two sets of constraints guarantee that each city is visited exactly once, whereas the subtour elimination constraints preclude disconnected cycles. The central challenge of CO is that its solution space combinatorially increases with the problem size, resulting in infeasible exhaustive enumeration for realistic instances [9,10]. The TSP and related routing problems have, therefore, become common benchmarks for neural constructive solvers, reinforcement learning policies, graph neural networks, and neural improvement heuristics [2,4].

Max-Cut is another graph-based example where an undirected weighted graph $G = (V, E)$ is given, and the goal is to partition V into two subsets so that the total weight of edges crossing the partition is maximized. Using spin variables $y_i \in \{-1, +1\}$, Max-Cut can be formulated as:

$$\max_{y \in \{-1,\, +1\}^{|V|}} \frac{1}{2} \sum_{(i,j) \in E} w_{ij}\left(1 - y_i y_j\right). \tag{4}$$

This equation is particularly important because it relates graph optimization to the Ising and quadratic unconstrained binary optimization (QUBO) models. Accordingly, Max-Cut and related graph problems are frequently used to evaluate quantum annealing, quantum-inspired algorithms, Ising machines, tensor network solvers, and specialized hardware for CO. QUBO or Ising formulations are useful in practice, but they rely on several factors. These factors are the solver speed, the cost of conversion between constrained and unconstrained forms, benchmark representativeness, embedding overhead, and classical baselines comparison [14].

Classical CO solution approaches are generally divided into exact algorithms, approximation algorithms, heuristics, and metaheuristics. Exact algorithm techniques, including branch-and-bound, branch-and-cut, cutting plane methods, dynamic programming, and constraint programming, seek to demonstrate optimality or infeasibility [9]. In minimization problems, the relative optimality gap can be defined by connecting the feasible and optimal solutions into one formula. Assume $\hat{x}$ is a feasible solution and x^* is an optimal solution; the relative optimality gap may be represented as

$$\text{Gap}\ (\hat{x}) = \frac{\mathrm{f}(\hat{x}) - \mathrm{f}\left(x^*\right)}{|\mathrm{f}(x^*)| + \varepsilon} \tag{5}$$

where $\varepsilon > 0$ is a small constant used to avoid division by zero. In case the optimum is unknown, solvers regularly give a primal dual gap utilizing the best feasible upper bound U and the best dual lower bound L as follows:

$$\text{PDGap} = \frac{U - L}{|U| + \varepsilon} \tag{6}$$

Such measures are extensively applied with studies involving mixed-integer programming, primal heuristics, solver warm-starting, and learning-assisted exact optimization [11,13]. In contrast, approximation algorithms search for polynomial time solutions that provide a proved performance guarantees for a chosen set of problem classes. For a minimization problem, an algorithm has an approximation ratio $\rho \geq 1$ if

$$\mathrm{f}(\hat{x}) \leq \rho \mathrm{f}\left(x^{*}\right) \tag{7}$$

For all instances in the target class [9,15]. Despite the substantial guarantee offered by the approximation theory, many real-world CO problems are still too large, constrained, dynamic, or domain-specific for exact or approximation methods alone.

As a result, this motivates the use of metaheuristics, including local search, simulated annealing, tabu search, evolutionary algorithms, ant colony optimization, genetic programming, and large-neighborhood search [1]. Machine learning contributes to CO by approximating, speeding up, or enhancing the mapping from a problem instance to a high-quality solution:

$$I \mapsto x^{*}(I) \tag{8}$$

In supervised neural CO, a model $\mathrm{p}_{\theta}(x \mid I)$ is trained using labelled solutions, often produced by exact solvers, expert-designed heuristics, or high-quality metaheuristics. Given a dataset $D = \left\{ \left(I_k, x_k^{*}\right) \right\}_{k=1}^{N}$, the training objective may be written as

$$\min_{\theta} \frac{1}{N} \sum_{k=1}^{N} \ell(\mathrm{p}_{\theta}(..|..I_k),\ x_k^{*}, \tag{9}$$

where $\ell(\cdot)$ is a supervised loss, such as cross-entropy or sequence prediction loss. In constructive neural solvers, a solution is often generated autoregressively:

$$\mathrm{p}_{\theta}(x|I) = \prod_{t=1}^{T} \mathrm{p}_{\theta}(a_t|s_t,\ I), \tag{10}$$

where a_t denotes the chosen action or decision at step t, s_t is the partial-solution state, and T is the construction horizon. Pointer networks and sequence-to-sequence architectures are considered early examples of this paradigm, especially for graph and routing problems.

Their major advantage lies in their ability to learn constructive patterns directly from data. Yet they may not generalize well when utilized for bigger or distributionally diverse scenarios, and they usually rely on high-quality labels for efficient training [2,4].

Another alternative learning model is reinforcement learning. In this model, CO is formulated as a form of sequential decision-making problem. The CO instance is structured as a Markov decision process (S, A, P, r, γ), where $s_t \in S$ is a partial solution or search state, $a_t \in A$ is a decision or improvement action, P is the transition rule, r_t is the reward, and γ is a discount factor. The goal is to maximize the expected return by learning a policy $\pi_{\theta}(a_t|s_t)$:

$$\mathrm{J}(\theta) = E_{\pi_{\theta}}\left[\sum_{t=0}^{T} \gamma^{t} r_t\right]. \tag{11}$$

In the context of minimization problems, the terminal reward is usually taken to be the negative objective value, $r_T = -\mathrm{f}(x_T)$, or as a gain relative to a baseline solution. Policy-gradient methods estimate is represented as follows:

$$\nabla_{\theta} \mathrm{J}(\theta) = \mathbb{E}_{\pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a_t|s_t)(R_t - \mathrm{b}(s_t))]. \tag{12}$$

where R_t denotes the return and b(s_t) denotes the baseline used to reduce variance. RL is appealing because it decreases reliance on optimal labels and naturally supports modeling constructive or improvement-based search. This explains its growing use in the TSP, VRP, knapsack, scheduling, virtual machine placement, flexible job-shop scheduling, resource allocation, and hyper-heuristic selection. In the reviewed studies, there are clear, consistent indications about sample inefficiency, reward shaping fragility, weak constraint handling, and limited generalization across instance sizes or distributions as persistent challenges [4].

A major foundation for learning-based CO has emerged from graph neural networks because many CO instances are naturally connected and relational. Usually, the problem instance is structured as a graph $G = (V, E)$, where nodes represent cities, jobs, machines, variables, constraints, facilities, customers, tasks, or graph vertices, while edges encode distances, precedence relations, compatibility, conflicts, or constraints [11,13]. The update of generic message passing GNN node embeddings in X is performed according to:

$$m_v^{(k)} = \bigoplus_{u \in \mathrm{N}(v)} \psi_\theta^{(k)} \left(h_v^{(k)}, h_u^{(k)}, e_{uv} \right). \tag{13}$$

$$h_v^{(k+1)} = \phi_\theta^{(k)} \left(h_v^{(k)}, hm_v^{(k)} \right). \tag{14}$$

where $h_v^{(k)}$ is the embedding of node v at layer k, e_{uv} is an edge feature, N(v) is the neighborhood of v, and $\oplus$ is a permutation-invariant aggregation function. The importance of this permutation equivariance lies in ensuring that the quality of the CO solution is independent of arbitrary node ordering. GNNs have been employed to estimate variable assignments, construct primal solutions, guide local improvement moves, learn graph representations for metaheuristics, solve unsupervised graph CO problems, and model bipartite variable–constraint graphs in MIP [11,13]. However, since X neural outputs are usually continuous, they must be encoded, rounded, repaired, or projected into feasible discrete solutions:

$$\hat{x} = \Pi_X(\mathrm{g}_\theta(G)) \tag{15}$$

where $\mathrm{g}_\theta(G)$ is the neural prediction and Π_X denotes a feasibility projection, repair operator, or solver-guided decoding mechanism. In neural CO, this projection phase continues to be a major challenge, particularly for issues with stringent limitations [11]. Another group merges hyper-heuristics and ML-enhanced metaheuristics. In such approaches, learning is used to enhance properties of search processes instead of replacing the optimizer. A learned model may predict promising neighborhoods, decide operators, predict variable importance, guide restore processes, tune parameters, warm-start populations, or model search dynamics [1]. This can be represented abstractly as

$$x_{t+1} = \mathrm{H}\left(X_t, \alpha_\theta(I, s_t)\right), \tag{16}$$

where H denotes a metaheuristic transition operator and α_θ denotes a learned control, selection, or prediction mechanism. A major benefit of this approach is that it maintains interpretability and modularity while integrating data-driven adaptation. However, the gain achieved is dependent on the problem domain, and comparisons are mostly constrained to the same metaheuristic without ML instead of against strong exact solvers, modern neural baselines, or well-tuned competing heuristics [1]. Another limitation addressed by predict-then-optimize and decision-focused learning is that, in many real-world CO problems, the optimization parameters are uncertain and require prediction from data. Let $\hat{c}_\theta(z)$ be a predicted cost vector from contextual features z. The downstream optimization problem is denoted as:

$$x^*(\hat{c}_\theta) \in \underset{x \in X}{\operatorname{argmin}} \hat{c}_\theta^T x. \tag{17}$$

Rather than optimizing prediction accuracy in isolation, decision-focused learning assesses the quality of the resulting decision under the true cost vector c. A common decision regret formulation is represented as follows:

$$\text{Regret}(\theta) = c^T x^*(\hat{c}_\theta) - c^T x^*(c). \tag{18}$$

The learning objective can then be written as

$$\min_\theta E_{(z,c)} \left[c^T x^*(\hat{c}_\theta(z)) - c^T x^*(c) \right] \tag{19}$$

This framework links model training to the quality of subsequent decisions rather than statistical prediction errors. However, because training could necessitate repeated calls to an optimization oracle or differentiating using a discrete argmin operator, it also presents methodological and computing challenges.

4. Literature Review and Critical Synthesis of ML-Based Combinatorial Optimization

Operations research, theoretical computer science, and numerous real-world decision problems, such as the traveling salesman problem, KP, Maximum Cut, Set Cover, MAX-SAT, scheduling, and vehicular route planning, all depend on CO. Exact solutions, approximation methods, and metaheuristics have long been used to handle these issues because many of them are NP-hard. DL and ML are currently complementary methods that can train solver modules, forecast possible solution structures, guide search, and replicate some aspects of the optimization process.

Figure 4 illustrates the role-oriented taxonomy that was employed in this study to categorize ML-for-CO strategies into seven methodological families according to the prominent function of learning in the optimization process. The taxonomy distinguishes methodological families based on the major role of learning in the optimization pathway while acknowledging that modern systems may incorporate "RL, GNNs, metaheuristics, exact solvers, and LLM-based" components. The crucial synthesis afterward is structurally based on this classification. This taxonomy serves as the structural foundation for the important synthesis that follows. This section is, consequently, divided into subject-based subsections, each of which offers a brief overview of the main concepts and a critical analysis of its advantages and limitations.

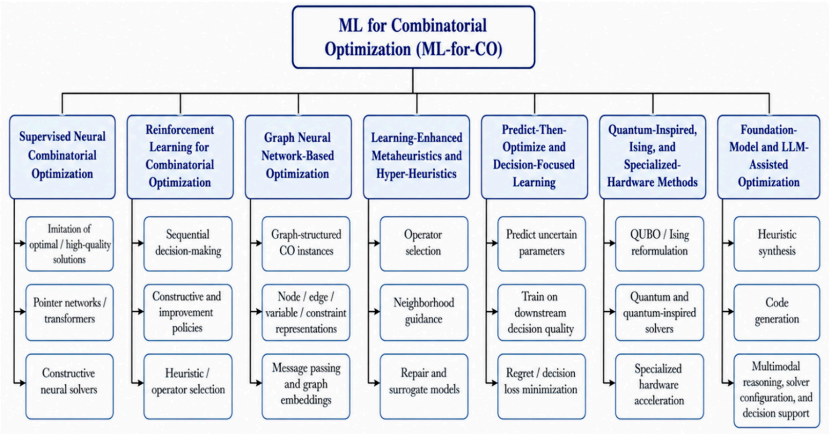


Figure 4. Role-based taxonomy of machine learning techniques for combinatorial optimization.

To eliminate theoretical overlapping within methodological families, the taxonomy is designed based on the main purpose of learning in the optimization pipeline instead of model design alone. Table 1 clarifies the boundaries between different taxonomy families.

Table 1. Boundary definitions for the ML-for-CO taxonomy.

Taxonomy Family	Definition Criterion	Boundary Explanation
Supervised neural CO solvers	Direct solution imitation/construction	Categorized here when labeled or near-optimal instances learning.
RL-for-CO	Reward-driven sequential decision-making	Categorized here when policy learning is essential.
GNN-based optimization	Graph/relational representation	GNNs may arise within RL or hybrid systems but are categorized here if representation is the primary contribution.
Learning-enhanced metaheuristics	ML enhances a current metaheuristic	The metaheuristic remains the primary solver.
Predict-then-optimize	Learning optimized for downstream decision quality	Whenever decision loss or regret is used to assess prediction.
Quantum/Ising methods	QUBO/Ising formulation or specialized hardware	The main contribution is the formulation and computing basis.
LLM-assisted optimization	Heuristic/code/model generation	LLMs enable optimization instead of providing certifications.
Hybrid neuro-symbolic optimization	Integration pattern	Not independent architecture; it integrates learning components and symbolic solvers.

4.1. Reviews and Fundamental Perspectives

The reviewed articles in this subsection fall into four categories: (i) versatile ML-for-CO surveys, (ii) paradigm-oriented reviews focusing on a particular learning strategy or subdomain, (iii) methodical related survey research (metaheuristics, Monte Carlo Tree Search (MCTS), multi-aspect decision-making), and (iv) domain-exclusive reviews highlighting where ML-for-CO is genuinely applied.

In the first group, the authors of [2] evaluate deep neural network alternatives to CO, categorizing techniques by network architecture (pointer networks, transformer-based encoders, GNNs) and learning concept (supervised, reinforced, unsupervised). Researchers in [1] adopt a combined strategy by concentrating on how ML complements metaheuristics, such as warm-starting populations, identifying good neighborhoods, or picking operators online. Reference [16] surveys optimization problems in ML, whereas [3] presents a comprehensive review of hyper-heuristics for CO. Article [14] provides an examination of physical hardware quantum solvers, while [17] explains the OR research area objectives.

A second group narrows the scope to specific learning models or problem types. In [4], they present a definitive investigation about RL utilization for CO, categorizing RL architectures by reformulating constructive, improvement, and hybrid across the TSP, VRP, KP, and scheduling. Researchers in [18] analyze ML-for-CO in energy relevance, including power consumption and unit devotion. Work in [5] provides an updated assessment of ML for improving metaheuristics in global optimization, supplementing [1] with an incessant domain perspective. In [6], they conducted a survey of solvers' learning techniques for CO in industrial manufacturing.

The third group focuses on methodological studies linked to ML-for-CO. In [19], they explore population-specific metaheuristics for massive unsupervised global optimization that is crucial for surrogate-assisted evolutionary optimization (SAEA) and augmented ML. The authors of [20] review the MCTS search methodology, which is directly associated with a RL-derived CO, including current algorithmic improvements and applications. The work in [21] examines heuristics and machine learning for optimizing multiple objectives in Small- and Medium-Sized Enterprise (SME) decision-making, delivering a realistic viewpoint on use in industry.

The fourth set includes particular surveys that show where ML-for-CO is currently used in practice. In [22], the authors presented a classification for AI-driven application

allocation in fog computing, whereas [23] surveyed resource allocation for IoT applications across edge–fog–cloud contexts. Both domains involve combinatorial decision-making problems such as deployment, scheduling, and packing. Graph CO issues in blockchain transaction network evaluation were investigated in [24]. In [25], researchers examined methods for predicting lead times in engineer-to-order settings, a scheduling issue with significant CO implications in the industry. Table 2 compares exemplary review and viewpoint publications based on their extent, CO issue types addressed, ML or optimization methodologies surveyed, main contributions, and important shortcomings.

Table 2. Summary of survey, review, and foundational studies on ML-for-CO.

Ref.	Review Scope	CO Problems/ Domains	ML/Optimization Methods Covered	Main Contribution	Critical Limitations
[1]	ML support for metaheuristics.	General CO, including TSP, VRP, scheduling, and packing.	ML-assisted operator selection; parameter tuning; fitness surrogates; configuration.	Defines major ML usage modes inside metaheuristic search.	Predates LLM/diffusion; limited robustness and deployment evidence.
[2]	DNN methods for CO.	TSP, VRP, JSP, KP, and Max-Cut.	Pointer networks; transformers; GNNs; supervised/RL/ unsupervised learning.	Provides an architecture vs learning paradigm taxonomy.	Primarily descriptive; non-unified benchmarks.
[3]	Systematic review of hyper-heuristics for CO.	Bin packing, scheduling, TSP, and VRP.	Selection/generation hyper-heuristics with RL, classifiers, regressors.	PRISMA-style synthesis of hyper-heuristic CO.	Pre-transformer/GNN era; limited scalability analysis.
[4]	Reinforcement learning for CO.	TSP, VRP, KP, JSP, and MIS.	Constructive, improvement, and hybrid RL with pointer/GNN policies.	Canonical RL-for-CO taxonomy; partial unified comparison.	Limited recent transformer/GNN-heavy coverage.
[5]	ML-enhanced metaheuristics in global optimization.	Continuous and combinatorial global optimization.	Operator selection; surrogates; hyperparameter learning	Recent ML+MH integration roadmap.	Continuous-optimization bias; CO implications sometimes indirect.
[6]	Learning to solve CO problems in manufacturing.	Scheduling, loT sizing, factory routing, JSP/FJSP.	Neural and RL manufacturing solvers.	Connect learning-based CO to industrial manufacturing needs.	Manufacturing-specific; limited LLM/diffusion/hybrid coverage.
[14]	CO on real quantum machines.	Max-Cut and Ising-formulated CO problems.	QAOA; quantum annealing on D-Wave/IBM-Q.	Practical quantum-hardware overview for CO.	Hardware-limited; narrow problem coverage.
[16]	Optimization problems arising inside ML.	SVMs, clustering, NN verification, compression.	MIP/QP; decomposition; convex relaxations.	Catalogues optimization-for-ML perspective.	Limited learning-to-optimize coverage.
[17]	ML-for-CO agenda.	General CO problems.	Position/editorial perspective.	Frames OR research agenda at ML-CO interface.	Brief; no empirical comparison or technical taxonomy.
[18]	ML-for-CO in energy applications.	Unit commitment, dispatch, OPF, EV scheduling.	Supervised learning; RL; GNNs.	Maps ML methods to power-system CO tasks.	Single-domain; limited cross-method comparison.
[19]	Large-scale black box optimization.	Large-scale continuous and some discrete benchmarks.	Decomposition; cooperative coevolution; EAs.	LSGO basis for surrogate-assisted search.	Continuous-domain focus; CO link is indirect
[20]	MCTS review.	Game tree search, planning, partial CO.	MCTS variants; neural-guided search.	Summarizes neural/search integrations.	Game/planning bias; limited CO-specific depth.
[21]	Strategic decision-making in SMEs with heuristics/ML.	Production, logistics, marketing mix.	MOO heuristics; ML decision support.	Practice-oriented MOO+ML evidence.	Industry-specific; limited transferability.

Table 2. *Cont.*

Ref.	Review Scope	CO Problems/ Domains	ML/Optimization Methods Covered	Main Contribution	Critical Limitations
[22]	Fog application placement.	Placement, assignment, scheduling.	ML; RL; metaheuristics.	Taxonomy of AI-based fog placement.	Fog-specific; CO formulation often sketched.
[23]	IoT edge–fog–cloud management.	Assignment, scheduling, offloading, and replication.	ML; RL; metaheuristics; classical heuristics.	Cross-tier resource-management taxonomy.	Architecture-centric; limited ML-for-CO comparison.
[24]	Blockchain graph CO.	Anomaly detection, clustering, matching.	Graph CO with ML and heuristic solvers.	Links blockchain analytics to graph CO.	Blockchain-specific; transfer unclear.
[25]	ETO lead-time prediction.	Industrial scheduling proxy.	ML regression; scheduling+ML hybrids.	Method taxonomy for ETO lead-time prediction.	Prediction-oriented; CO role is downstream.

Strengths and limitations:

The surveys and foundational research analyzed in this section present a significant theoretical base for ML-for-CO by developing a standard terminology that includes terms such as "end-to-end" learning and "learning-to-configure" optimization. They are especially useful when they relate neural architecture to metaheuristic and solver-oriented models, such as in [1,2,4,5], allowing for comparability within disparate methodological categories. The surveys further emphasize significant domain-specific trends, namely, the shift from supervised mimicking to RL and from only relying on neural solvers to hybrid neuro-symbolic optimization systems. Furthermore, field-specific reviews [6,18,22–25] extend the idea by showing applied adoption in energy, fabrication, fog/IoT, blockchain, and engineer-to-order domains, which are frequently overlooked in solely methodological reviews.

Despite the positive attributes described above, there are certain shortcomings and unaddressed methodological issues, which are listed as follows. The research literature's main shortcoming is its fast-outdated nature. Studies produced between 2020 and 2022 precede the current LLM-for-optimization trend depicted in [7,8,26,27], while the recent surveys [5,6] only minimally cover LLM-based heuristic composition. Benchmarking is also ambiguous, as few surveys apply consistent assessment methodologies; thus, cross-paper statements on cutting-edge performance should be evaluated with caution. The study by [4] is a bare exception since it provides a more unified evaluation of RL-based approaches. Aside from Gambella et al. [16], most review articles are descriptive rather than analytic, with limited "formal complexity", "sample complexity", or generalization outcomes. Lastly, numerous sector-specific reviews [18,22–25] investigate CO-relevant ML primarily within their respective application areas, with little integration into the larger ML-for-CO methodological literature, potentially leading to repetition and minimal interchange of concepts and insights among research areas.

4.2. Methods of Reinforcement Learning for Combinatorial Optimization

The reward-oriented sequential decision-making structure of RL-for-CO sets it apart from supervised neural solvers in this review. Instead of directly mimicking annotated optimal solutions, the model learns policies via association with an optimization context. The transfer of discrete restrictions to consecutive Markov decision processes (MDPs) is the foundation for the usefulness of RL in CO. This structure makes it possible for value-driven and gradient-based policy models to build solutions using repetitive activities, like resource scheduling or TSP pathways. RL offers a scalable approach for applying heuristics in computationally exorbitant NP-hard fields by optimization with reward indicators instead of ground-truth labels. This trend is exemplified by a number of papers in this list that

span remarkably different application domains. While [28] focused on sustainable virtual machine (VM) assignment, reference [29] created scalable RL for VM rescheduling in cloud data centers. For linked transportation manufacturing scheduling, the researchers of [30] used a multi-faceted diversity-based quality methodology with a Deep Q-Network. For adaptable job market scheduling, the work in [31] combined multiple-agent deep RL (DRL) with "constraint programming". The methodological pedigree was demonstrated by previous comparable studies on Deep Q-Network (DQN) selection hyper-heuristics [32] and Max-Cut with "pointer networks" under supervised learning integrated with RL [33]. The application of Q-learning within a tri-stage proactive algorithm for decentralized adaptable job market scheduling with worker factors was also shown by the authors of [34]. This is an example of the broader shift to RL as a component rather than RL as the entire solver. The problem area, learning strategy, model structure, assessment metrics, findings reported, and major limitations of RL-based CO investigations are summarized in Table 3.

Table 3. Comparative analysis of RL approaches for combinatorial optimization.

Ref.	CO Problem/ Domain	RL Formulation	Model/Solver Integration	Evaluation Metrics/Baselines	Critical Limitations
[28]	Sustainable VM placement	Sustainability-aware placement policy	RL policy for power/SLA/utilization balance	Power, SLA, utilization vs heuristics	Environment-specific objective; weak DRL/CP baselines
[29]	VM rescheduling	Dynamic resource-allocation policy	Scalable deep RL rescheduling architecture	SLA, energy/cost, makespan vs heuristics	Production-specific; reward transfer uncertain
[30]	Transportation-production scheduling	DQN-assisted multi-objective search	DQN inside QD/MOEA loop	Hypervolume, IGD vs MOEAs	Many hyperparameters; unclear many-objective scalability
[31]	Flexible job-shop scheduling	Multi-agent DRL with feasibility filtering	CP-filtered DRL actions	Makespan, feasibility rate	CP filtering cost; FJSP-specific evaluation
[32]	Selection hyper-heuristics	Online heuristic-selection policy	DQN selects low-level heuristics	Best objective on benchmarks	Sample inefficient; weak regret/optimality guarantees
[33]	Max-Cut	Supervised + policy-gradient RL	LSTM pointer network decoder	Cut value, optimality gap	Sequential decoding limits scale; pre-transformer design
[34]	Multi-objective distributed FJSP	Q-learning search control	Q-learning in tri-stage cooperative optimizer	Hypervolume, IGD, makespan	Worker factor specificity; limited transfer
[35]	Dynamic flexible scheduling	DRL composes symbolic GP actions	GP + DRL hybrid	Mean tardiness, makespan	Hand-designed action space; interpretability unverified
[36]	Underwater acoustic networks	Hierarchical combinatorial bandit	Bandit/RL for joint resource allocation	Throughput, energy efficiency	Stationarity/channel assumptions may fail

Strengths and limitations:

Since RL can learn immediately from exposure to the optimization conditions, it reduces reliance on costly best-solution labels and provides a label-aware substitute for supervised neural CO. The methodological adaptability of RL conceptions is further demonstrated by the adaptation of "pointer networks", "attention-based encoders", "Q-learning mechanisms", and related policy-based learning methods to a variety of route planning, scheduling, packing, and resource allocation challenges. As shown in [30,31,34,35], recent research also points to a trending hybrid RL design, where learning is coupled with "constraint programming", "evolutionary search", or genetic algorithms to increase practicality and lessen the weakening of solely neural policies.

Despite these advantages, there are significant restrictions and unsettled methodological issues, which are next illustrated. Although it is adaptable, RL-for-CO is still

computationally costly, as many solvers require substantial simulations and huge numbers of training trials, resulting in issues regarding the cost of training and reproducibility. Generalization is also constrained as policies taught on a specific case scale or distribution, including the TSP-100, frequently worsen when applied to bigger or fundamentally diverse cases, while "curriculum learning" and "meta-learning" offer partial solutions, as indicated in [37]. Reward formulation represents a significant difficulty, as the scarcity of endpoint objectives—such as makespan or feasibility—makes performance highly susceptible to heuristic molding. Also, these molding mechanisms are routinely used with no strict, methodical ablation to separate their quantitative significance. Lastly, standalone RL strategies frequently fail to meet strict limits, which supports the growing usage of hybrid RL-CP approaches, such as [31,38]. To conclude, sample inefficiency, reward sensitivity, and weak constraint satisfaction are the critical limitations of RL-based CO methods.

The abovementioned RL limitations addressing constraints relatively inspire the growing employment of GNNs.

4.3. Graph Neural Networks and Deep Learning Architectures

For CO issues with inherently relational instances, GNNs offer the prevailing structural constraints if RL affords the learning model. The adoption of graph, variable-constrained, node–edge, or relational structures to encode CO architecture is what distinguishes GNN-based approaches, which are considered in this review as a representation-oriented family. The reviewed studies in this section make the case that message-transferring architectures constitute a medium that is almost universal. For the limited progressive graph drawing problem, Reference [13] used mixed graph modeling with metaheuristics. The authors of [39] address CO under heterophily, a notably understudied system, and reference [34] suggested a metaheuristic GNN embedding search. The researchers in [40] created neural refinement heuristics, and reference [36] created scaling primitive heuristics using GNNs. Research in [41] presented a two-level framework for CO on graphs, while reference [42] focused on unsupervised CO under cardinality and coverage constraints.

While ML-enriched metaheuristics keep traditional search structures and are frequently more deployable than RL-based solutions, GNN-based approaches offer higher relational inductive bias but still require feasibility projection. Neural CO and typical GNN approaches are contrasted in Table 4.

Table 4. Graph neural networks and DL architectures for combinatorial optimization.

Ref.	CO Problem/Domain	Graph Representation	Learning Architecture	Evaluation Metrics/Findings	Critical Limitations
[11]	MIP primal heuristics	Variable-constraint bipartite graph	GNN predicts primal assignments	Primal gap/integral; faster selected MIPLIB closure	Uneven MIP family transfer; weak search integration
[13]	Constrained incremental graph drawing	Problem graph features	Graph representation + metaheuristics	Crossing number; runtime	Problem-specific; limited graph-CO transfer
[39]	Heterophilic CO: MIS, Max-Cut	Heterophilic graph signals	Heterophily-aware GNN	Approximation ratio	Promising but narrow CO-family evidence
[41]	Graph CO	Graph instance representation	Bi-level GNN-guided solver	Solution quality on graph benchmarks	Costly bi-level training; no convergence guarantees
[40]	Graph CO: TSP/VRP	Route/problem graph	GNN neural improvement heuristic	Fixed-budget solution quality	Initial solution dependence; weaker on constrained cases
[42]	Cardinality/covering constraints	Constraint graph/relaxation	Unsupervised GNN surrogate	Approximation ratio; constraint violations	Problem-specific relaxations; discrete continuous gap
[43]	CO solver architecture search	Encoded GNN architectures	Metaheuristic-guided NAS	Optimality gap; wall-clock search	High NAS overhead; uncertain cost-benefit

Strengths and limitations:

The main contributions and methodological benefits are as follows. Because their order-independent information-sharing structure respects the intrinsic symmetries of graph-structured CO instances, graph neural networks (GNNs) offer a solid architectural bias for CO, which prevents the autonomous network members or progressions imposed by previous pre-GNN models. Trained GNN heuristics often show greater generality across case sizes than merely sequential designs when combined with on-device inference, facilitating more effective meta-learning. The prospect of unsupervised or barely supervised GNN training has also been shown in recent work [42]. Additionally, researchers in [39] pointed out that effective CO strategies can be learned independently of optimum labels, which is a significant benefit as the instance scale grows and precise labels become unaffordable.

Despite these benefits, there are still a number of restrictions and unsolved methodological issues. In heavy or architecturally complicated problems, like MIS and Max-Cut, deep GNNs might experience problems, like excessive smoothing, where repetitive data transfer diminishes classification effectiveness and restricts its capacity to identify dependencies over time. Although the work in [39] is a noteworthy rare exception, the majority of GNN-based solvers still presume a same-label-adjacent graph architecture; different-label-adjacent CO cases are still not well studied. Furthermore, standard GNN outputs are continuous embeddings that need to be translated into viable discrete solutions, decoded, rounded, or rectified; yet this practical enforcement phase is frequently seen as a subsidiary instead of a fundamental procedural component. Lastly, there is a significant computational burden associated with neural architecture search (NAS) techniques, like [43], and it is still unknown if the performance improvements they provide regularly outweigh the increased search expense. In brief, over-smoothing, heterophily, feasibility projection, and decoding cost are the critical limitations of GNN-driven CO methods.

Since GNNs present architectural enhancements, there remains a need for mechanisms for decoding and validity, which motivates the use of hybrid solver integration.

4.4. Quantum, Quantum-Inspired, and Ising Machine Methods

A significant portion of the examined works use Ising/QUBO mathematical models implemented on particular quantum or quantum-driven hardware for approaching CO. Researchers described hybrid short-term photonic quantum computers in [44], and they categorized and compared QUBO-based quantum annealing algorithms in [45]. The authors of [46] assessed conventional heuristics driven by quantum annealing, while the authors of [47] presented non-Boolean Ising optimization. A tree search technique over Ising formulations was developed by researchers in [48], and PyQUBO, the de facto Python 3.9 library for transforming CO to QUBO, was made available in [49]. Furthermore, the authors of [50] investigated warm-started QAOA hyperparameter selection on Max-Cut, and the study [51] investigated quantum-based tensor networks for restricted CO. Recurrent Neural Networks (RNNs) were integrated with annealing in [52]. Lastly, ReRAM- and GPU-powered Ising machines were created by the work in [53,54]. The examined studies in this section are compared in Table 5.

Strengths and limitations:

QUBO and Ising mathematical models provide a consistent, problem-independent framework for handling a range of CO domains. Mapping from constrained formulations to solver-like models can be standardized with the use of tools such as PyQUBO [49]. Such an abstraction has enabled experiments with "quantum annealing", quantum-driven algorithms, and tailored Ising hardware. Recent hardware-oriented research also suggests promising speeds on particular QUBO instances, such as ReAIM [53] and GPU-powered Ising machines [54]. Concurrently, quantum-driven classical methods [46] are useful

because they can often match or outperform current quantum setups while still being deployed on traditional computer infrastructure.

Table 5. Quantum-inspired, Ising, and specialized hardware approaches for CO.

Ref.	CO Formulation	Computing Paradigm	Solver/Platform	Evaluation Metrics/Findings	Critical Limitations
[44]	CO/ML mapped to quantum routines	Hybrid quantum classical	Photonic quantum hardware	Time-to-solution; photon count	Small instances; narrow classical comparison
[45]	QUBO NP-hard formulations	Quantum annealing	D-Wave annealer	Time-to-solution; success probability	Minor embedding overhead; limited classical baselines
[46]	Generic QUBO/Ising instances	Quantum-inspired classical optimization	Simulated bifurcation/quantum annealing	Solution quality vs quantum annealing	No demonstrated quantum advantage
[47]	Max-Cut/Ising CO	Dynamical Ising machine	Continuous non-binary spin solver	Cut value; runtime	Basin escape theory incomplete
[49]	Constrained CO to QUBO	QUBO software tooling	PyQUBO + downstream solvers	Usability; mapping performance	Not a learning method; penalty/solver dependent
[50]	Warm-started Max-Cut QAOA	Variational quantum optimization	QAOA hyperparameter optimization	Approximation ratio at low depth	Max-Cut only; hardware/simulation size limits
[52]	Ising/CO instances	Neural variational + annealing	RNN ansatz + simulated annealing	Approximation ratio	Limited strong annealing/GNN baselines
[53]	Ising/QUBO solving	Specialized analog hardware	ReRAM adaptive Ising machine	Energy/time per solution	Platform-specific; low reproducibility

The main drawbacks of ML and quantum-powered approaches are as follows. The primary obstacle is the overhead costs of transforming restricted CO issues into unlimited QUBO format, which often requires penalties that could distort the optimization and slow down search speed. In addition, quantum annealing methods require modest representation, which can scale poorly with graph depth and is not always properly handled in performance claims; many claimed quantum advantage or hardware acceleration results rely on small, managed, or selectively chosen benchmarks, and comparisons with robust classical solvers, like Gurobi or KaMIS, are still uncommon. Lastly, because it is challenging to duplicate studies on photonic, Re-RAM, FPGA, or other particular platforms regardless of access to the same hardware setting, reproducibility is a recurring challenge. Overall, QUBO mapping overhead, hardware embedding, and reproducibility are the critical limitations of quantum/Ising CO methods.

Although different computational representations for CO are provided by quantum-inspired and Ising-driven approaches, their concrete shortcomings in mapping, embedding, and reproducibility highlight the significance of more deployable metaheuristic and hyper-heuristic techniques, especially those that improve current metaheuristic search with learned components.

4.5. ML-Enhanced Metaheuristics and Hyper-Heuristics

In a line of research that is both pragmatic and significant, ML-for-CO aims to complement pre-existing metaheuristic frameworks with learnt components rather than supplanting the fundamental search procedure. In contrast to end-to-end neural solvers, this category uses ML to enhance certain aspects like operator selection, repair, neighborhood guiding, surrogate assessment, or parameter control, while the metaheuristic continues to be the primary optimization engine. This involves integrating a binary ML classifier into Cuckoo Search for the Set-Union Knapsack Challenge [55], establishing a binary

Dream Optimization algorithm, along with data-oriented correction for the Minimum Cost Coverage Problem [56], and implementing a mentor training dynamic search for the "Weighted Independent Set Problem" [57]. Other contributions investigate variable significance predictions for multi-dimensional optimization [58], whether machine learning helps pallet loading optimization [59], and the selection of autonomously created heuristics for the Block Relocation Problem [60]. Further research suggests using learnt Massive Neighborhood Searching for naval cargo route planning [61], dynamic geometry-driven meta-learning for multi-objective CO [37], and ML-based modeling of TSP backtracking effort [62]. The literature on machine learning-enhanced metaheuristics provides a broader conceptual underpinning for this category [1]. Table 6 compares approaches that embed ML components into metaheuristic or hyper-heuristic frameworks.

Table 6. ML-enhanced metaheuristics and hyper-heuristics for CO.

Ref.	CO Problem	Metaheuristic Backbone	Learned Component/ML Role	Evaluation Metrics/Findings	Critical Limitations
[55]	Set union KP	Cuckoo search + local search	Classifier-guided candidate selection	Best/average objective	No modern exact baselines; dataset-specific tuning
[56]	Set covering problem	Binary dream optimization	Data-driven repair	Best objective on OR-Library SCP	Weak theoretical novelty; close baseline gains
[57]	Weighted independent set	Adaptive local search	Learning-guided search	Solution quality; runtime	WIS-limited; no broader graph-CO transfer
[58]	High-dimensional black box optimization	Surrogate-assisted search	Variable-importance estimation	Convergence speed; active variables	Surrogate-dependent; costly for black box settings
[59]	Pallet loading	Classical loading heuristics	ML vs classical heuristic comparison	Filling ratio; runtime	Useful mixed result; limited generality
[60]	Container relocation	Automatically designed GP heuristics	Selection among evolved heuristics	Number of relocations	Synthetic training; OOD performance unclear
[61]	Maritime inventory routing	Large neighborhood search	Learned neighborhood selection	Cost gap vs MIP; runtime	Private data; limited independent validation

Strengths and limitations:

ML-enhanced metaheuristics offer a useful method to boost combinatorial search without altering the core solver. To increase search effectiveness and solution quality, learned elements, including operator selectors, area-level predictors, variable significance estimators, meta-models, and repair mechanisms, can be added to already-existing metaheuristic frameworks. Because the metaheuristic backbone is still traceable and interpretable, which is crucial for industrial adoption, this flexibility has practical value. Also, learned modules can often be trained again for specific case categories while keeping the broader algorithmic framework.

There are still some limitations that require researchers' attention as follows. When compared to well-tuned baselines, the empirical gains of ML-enhanced metaheuristics are frequently modest, and purported improvements are not always supported by statistical significance tests or multi-seed trials. Additionally, a lot of research relies on private industrial datasets, specialized fixed operators, or a problem-based feature design process, which reduces consistency and undermines assurances of generalization. A comparison metric weakness is another issue. ML-assisted metaheuristics are often tested against their non-ML counterparts but not robust exact solvers, like Gurobi or CPLEX, contemporary predictive solver models, or well-tuned rival heuristics. Overall, marginal gains, weak baselines, and problem-specific engineering are the critical limitations of metaheuristic CO methods.

Unlike ML-enriched metaheuristics, which mainly enhance the searching mechanism, "predict-then-optimize" and "differentiable learning" methods tackle a different problem: connecting predictive models with downstream optimization quality.

4.6. Predict-Then-Optimize and End-to-End Differentiable Pipelines

Combinatorial optimization is viewed as the downstream phase of a learning process in which underlying models predict ambiguous issue parameter values, like costs, requests, or capacity, according to a theory-relevant thread. As such, explicit generalization limitations for the "predict-then-optimize" scheme have been devised [63], and "Smart Predict-and-Optimize (SPO)" has been adapted to tough CO issues [12]. Additionally, the data creation problem—that is, how to create valuable training examples when the best solutions are costly to acquire—has been studied [64]. An effective perceptron architecture for NP-hard CO [65], CombOptNet for "end-to-end learning" of integer-based programming bounds [66], dynamic solution predictability [67], learning under tough linear conditions [68,69], and learning MAX-SAT computations from context-specific instances [70] are further contributions. Neuralizing Message Passing in MAX-E-3-SAT [71], Gumbel–Softmax optimization for graph-derived CO [72], and pointer network techniques for unbounded binary quadratic programming have all been further investigated in related work [73]. The decision-driven learning strategies discussed in this section are contrasted in Table 7.

Table 7. Predict-then-optimize and end-to-end differentiable learning pipelines for CO.

Ref.	CO Problem	Prediction/ Learning Target	Optimization Oracle/Strategy	Decision Metric/ Findings	Critical Limitations
[12]	Hard CO: matching, KP	Unknown downstream parameters	SPO+ surrogate + optimization oracle	Regret vs two-stage learning	Loose surrogate risk; oracle call cost
[63]	Predict-then-optimize	Decision-focused generalization	Statistical learning/Rademacher analysis	Sample complexity bounds	Bounds may be loose; solver assumptions idealized
[64]	Learning hard optimization	Training instance generation	Active sampling	Solution quality vs random sampling	Expensive and learner-dependent sampling
[66]	Learning IP constraints	Constraint recovery	Black box differentiation through ILP	Constraint recovery; decision accuracy	Poor scaling with constraints; sensitive differentiation
[67]	MIP solution prediction	Warm-start solutions	Adaptive predictor + exact solver	Time-to-optimality; primal gap	Instance similarity dependence; weak shift analysis
[68]	Hard linear constraints	Feasible decision prediction	Differentiable projection	Violation rate; decision quality	Linear only; no integrality/nonlinear feasibility
[70]	Contextual MAX-SAT	Hidden formula/constraint learning	Constraint learning from examples	Recovered constraint F1; MAX-SAT score	High sample complexity; feature space dependence
[72]	Graph CO: Max-Cut, MIS	Continuous discrete decision relaxation	Gumbel–Softmax relaxation	Solution quality	Relaxation gap; temperature sensitivity

Strengths and limitations:

Here are the main strengths of the reviewed articles' methodologies. "Predict-then-optimize and end-to-end differentiable" pipelines are within the ML-for-CO components with the best conceptual foundations. The researcher in [63] provided unusual formal generalization requirements, whereas the researchers in [12,66] provided well-reasoned distinct surrogate models for linking forecasting techniques with cascade optimization. One important advantage of this particular research category is its degree of decision integrity. Decision-centered and Sensible Predict-and-Optimize (SPO)-style methods can increase

performance under model ambiguity by improving the final decision loss rather than only reducing prediction error. With CombOptNet [66,70] tackling the understudied issue of learning restrictions derived from data, these research efforts further expand ML-for-CO beyond rule-based learning, which can be considered an important advance for OR that is more data-centric.

Although it is still technically difficult to determine the minimal operators via discrete argumentation, surrogate modal relaxations, like black box gradient approximation, perturbation and mapping, or Gumbel–Softmax, are often required [72], which may result in bias-driven gradients, substantial variance, or relaxation gaps. Another significant limitation is computational expense because training frequently necessitates multiple calls to optimization oracles, which limit scaling over small instance numbers. As they noted in [64], generating high-quality or optimal training instances can be a challenging CO problem in and of itself. Data production is still a concern. Lastly, decision-centered models learned on one cost or instance distribution might fail when implemented under various operating situations since distribution shift has not been thoroughly studied. In closing, oracle cost, relaxation bias, and distribution shift are the critical limitations of predict-then-optimize CO methods.

While decision-based learning offers a logical connection between optimization and prediction, its usefulness is best evaluated in specific domain contexts where deployment goals, operational limitations, and uncertain variables interact.

4.7. Domain-Specific Applications

A significant amount of current ML-for-CO research is driven by application-centric demands, where learning techniques are assessed under operational goals and field-specific limitations. CP and ML are combined in the healthcare industry to help with scheduling choices [38]. Predictive modeling facilitates tensile strength prediction in alloys with high entropy [74], combinatorial inkjet printing and machine learning are utilized for "Rare-Earth Barium Copper Oxide (REBCO)" superconductor thin films [75], and AI-powered optimization is used in a particle-physics experiment design [76]. "Black-box" optimization is studied for "Radio Access Network (RAN)" function deployment in networks and communication systems [77], and learning-driven approaches are applied for computation migration in decentralized learning and vehicle contexts [78,79]. A proxy-aided sim heuristic is suggested for hotel dynamic price adjustment in service operations [80]. These research investigations show how ML-for-CO can be applied in a variety of fields, including medical care, physics, materials research, communication technology, vehicle edge computation, and management of revenue, as shown in Table 8.

Strengths and limitations:

Field-specific studies provide crucial evidence that ML-for-CO can manage practically applicable optimization problems in cloud computing, communications, healthcare, materials science, and physics. Integrating genuine operational restrictions, such as nurse schedule rules, RAN structure, alloy phase behavior, and particular domain re-sourcing limitations, which are frequently oversimplified in merely conceptual investigations, is just as valuable as their factual performance. Specifically, applications related to materials science [74,75,77] show how combinatorial design may be combined with "Bayesian optimization" and active learning, extending the methodological reach of ML-for-CO beyond traditional route planning, scheduling, and graph benchmarks.

There are still some shortcomings in the current domain-specific CO research efforts as follows. Several domain studies rely on well-known ML resources rather than inventing the basic ML-for-CO technique, and they often have to pick between methodological flexibility and practical applicability. Because a model created for cloud scheduling, for instance,

is difficult to apply straight to medical scheduling without significant modification, its outcomes are also prone to poor transfer between domains. Heterogeneous assessment metrics, such as energy effectiveness, throughput, makespan, "Inverted Generational Distance (IGD)", and (R^2) for strong yield estimates, complicate cross-check comparability even more. Lastly, because effective uses are more likely to be published than unsuccessful deployments, unfavorable outcomes, or situations where ML offers little help, the literature is susceptible to publication and selection bias.

Table 8. Domain-specific applications of machine learning for CO.

Ref.	Application Domain	CO Task	ML/Optimization Method	Metrics and Main Contribution	Transferability Limitations
[38]	Healthcare	Institutional scheduling	ML predictor + CP	Feasibility, makespan, fairness	Hospital-specific rules limit transfer
[74]	High-entropy alloys	Composition-property search	Composition regressor	R^2, RMSE for yield strength	Limited by training composition coverage
[75]	Materials/ superconducting films	Combinatorial materials screening	Inkjet printing + ML analysis	Material yield; screening efficiency	Small libraries; chemistry-specific generalization
[76]	Particle physics	Experiment design	Bayesian/RL design	Information gain; sensitivity	Highly domain-specific
[77]	Telecommunications/ RAN	Function placement	BO, CMA-ES, RL comparison	Latency; deployment cost	Small, nonstandard benchmarks
[78]	Vehicular edge computing	Task prioritization/offloading	Counterfactual multi-agent DRL	Latency, energy, fairness; ~6% lower latency	Scalability, mobility, reward, delay assumptions
[79]	Vehicular networks	Offloading/resource allocation	Learning-based joint scheduling	Delay, energy, drop ratio	Simplified mobility/channel models; limited reproducibility
[80]	Hospitality/revenue management	Hotel dynamic pricing	Simulation + ML + metaheuristics	Revenue/RevPAR uplift	Demand assumptions constrain generality

The variety of domain-dependent applications shows ML-for-CO's realistic scope, but it also highlights unanswered concerns regarding robustness, transferability, and reliability, which call for a closer examination of formal limitations and theoretical underpinnings.

4.8. Theoretical Foundations and Robustness

The conceptual basis of learning for CO and the resilience of learnt algorithms under aggressive or heuristic inadequacies are examined in a narrow but conceptually significant fraction of the literature. Establishing proof of lower boundaries for random occurrences problems [81], hardness augmentation in optimization [82], and the development of challenging cases for effective CO evaluation [83] are examples of this field of work. The literature also comprises research efforts that question whether multi-faceted problems are genuinely tough in reality, implying that the complexity is not general but is contingent on certain assessment approaches [84]. Others provided structured rationalization for different CO techniques [85]. In [86], they discussed the AI integration into CO algorithms, and the study conducted in [87] presented optimization of hyperparameters via combinatorial methods. Concepts reflect contemporary, intelligent approaches to solving NP-complex, uncertain, or highly difficult computational problems by merging long-established mathematics with data-driven AI, which were demonstrated in studies [88–90]. These works offer a conceptual and rigorous counterpoint to the primarily practical ML-for-CO literature. Table 9 contrasts the methodologies discussed in this section.

Table 9. Theoretical, robustness, and explainability-oriented studies for learning-based CO.

Ref.	Theoretical Focus	CO Setting	Methodological Lens	Formal/Empirical Output and Relevance	Critical Limitations
[81]	Low-degree hardness	Random optimization	Statistical physics; complexity	Lower bounds; limits efficient learning	Random/worst-case setting may not match industry
[82]	Hardness amplification	Optimization reductions	Complexity-theoretic analysis	Hardness magnification factors	Mostly theoretical; limited design guidance
[83]	Hard-instance generation	Robust CO	Adversarial instance generation	Runtime and robustness-gap stress tests	Problem-specific generator; not a learning method
[84]	Many-objective CO difficulty	Many-objective CO	Metric/decomposition analysis	Hypervolume/ decomposition evidence	Metric-dependent; provocative, not universal
[85]	CO explanations	Explaining CO solutions	Logical explanation framework	Explanation size; faithfulness; XAI link	Limited empirical integration with neural CO
[88]	Learning for robust CO	Optimization under uncertainty	Deep learning + robust optimization	Worst-case and average regret	Uncertainty-set dependence; narrow coverage

Strengths and limitations:

By shifting the emphasis from actual performance to analysis-based justification, the studies reviewed in this part provide a theoretical basis for ML-for-CO. They reveal the consistency, understanding, and basic bounds of neural solvers by basing learned optimization on intricate computations, allowing evaluation against acknowledged computational constraints rather than isolated benchmark achievements. [81,82]. Resilience-focused research shifts the priority from typical-case benchmark performance to dependability under uncertain conditions, hostile instance formation, and deployment-specific robustness [83,88]. Plus, explanatory work relates CO to larger explainable AI research, emphasizing the significance of understandable solution reasoning in high-level optimization contexts [85].

The main limits and issues regarding the conceptual foundation of CO problems are listed next. The discrepancy between theoretical findings and real-world ML-for-CO deployment is a persistent challenge. Hardness assessments usually concentrate on extreme cases or random problem patterns, while real-world examples usually have exploitable structures. Some theoretical statements must also be carefully interpreted; for instance, claiming that multi-faceted CO is "easy" is very dependent on the performance measures and breakdown assumptions used [84]. Lastly, the rapid methodological use of theoretical and robustness-driven investigations in mainstream ML-for-CO research is limited since they are rarely backed by repetitive benchmark processes or expandable empirical proof.

The constraints and reliability needs of learnt solvers are clarified by conceptual and robustness-centered investigations, but the current development of LLMs brings a novel perspective that is more concerned with heuristic synthesis, code generation, and solver help than direct optimization.

4.9. Large Language Models for Combinatorial Optimization

The use of large language models (LLMs) in combinatorial optimization is the latest conceptual shift in this field. Since LLMs are mostly used to create, improve, or modify optimization processes rather than to learn an established task-specific solver from scratch, LLM-aided CO is distinct from traditional neural solvers. Current studies use LLMs to discover heuristics for mixed-integer scheduling [7], support multimodal reasoning for vehicle routing [8], guide heuristic evolution for diverse VRP variants [26], and generate efficient heuristics for broader CO problems [27]. These studies show a deliberate shift

away from creating highly specialized neural solutions from scratch and toward the generation, evolution, or refinement of optimization processes utilizing foundation models. In this context, LLMs can reason over routing or scheduling contexts, facilitate heuristic-code creation, and direct classical or neural solver components. However, since LLM-based optimization still needs stringent verification, contamination-resistant benchmarks, repeatable model methods, and cost-normalized comparison to conventional, metaheuristic, and neural baselines, this is still a promising and risky research avenue. Table 10 compares the surveyed studies in this section.

Table 10. Large language model-based approaches for CO.

Ref.	CO Task	LLM Role/Modality	Output/Verification Mechanism	Metrics/Baselines	Critical Limitations
[7]	MIP single-machine scheduling	Text-based heuristic/code generation	Generated heuristics checked by MIP solvers	Optimality gap; speed up	Closed-model dependence; contamination; reproducibility
[8]	Vehicle routing	Multimodal map/text reasoning	Routes verified by classical VRP solvers	Tour cost vs VRP baselines	High inference cost; no certificates; small instances
[27]	General CO heuristic generation	Core abstraction prompting; fitness prediction	Generated code/fitness checked against HG baselines	Multi-task performance; reduced evaluation cost	Prompt/model sensitivity; weak unseen transfer
[26]	Diverse VRP variants	LLM-guided heuristic evolution	Evolved solver components tested on VRP variants	Routing quality; cross-variant performance	Benchmark-dependent generalization; evolution cost

Strengths and limitations:

Large language models (LLMs) offer a flexible model for ML-for-CO since they can tackle various problem variations with little or no task-oriented retraining, allowing zero- and few-shot adjustment beyond the capability of traditional neural solvers. Their principal significance is in the automated development and refinement of problem solving solutions. Basically, these techniques produce or adapt putative rules of thumb (heuristics), which are then exhaustively examined for efficacy using either classical mathematical methods or current ML models [7,26,27]. LLMs further broaden the methodological scope of CO through multimodal grounding, in which visual map information may be included in routing-oriented reasoning, implying the possibility of deeper real-world optimization settings [8].

No matter how flexible they are, LLM-derived heuristics and solutions rarely produce superiority or feasibility certifications; therefore, their performance needs to be verified via benchmark analysis, problem-oriented feasibility tests, or traditional solvers. Cost and latency remain major obstacles, especially when large instance batches require numerous LLM calls. Another significant issue is data contamination, since pretraining corpora may contain standard CO benchmarks, undercutting claims of generalization. Lastly, since frozen-model evaluation methods have not yet been established in the field and closed-source LLMs can differ between releases, reproducibility is restricted. In conclusion, verification, cost, contamination, and reproducibility are the main limitations of LLM CO methods.

As a whole, the examined approaches demonstrate that ML-for-CO is moving toward hybridization instead of a standalone learning model, with learnt models frequently acting as elements that reinforce, speed up, or guide traditional optimization foundations rather than completely substituting them.

4.10. Research Gaps

A citation-supported evolving taxonomy of ML-for-CO techniques from 2010 to 2026 is shown in Figure 5. It illustrates the evolution of the discipline from pre-DL classical optimization foundations to four main methodological streams: theory/robustness/explainability, neural CO solvers, learning-assisted optimization, and quantum and alternative computing. Additionally, it represents LLM-aided CO as a new field that focuses on code creation, structured reasoning, and heuristic synthesis. Crucially, rather than being seen as a distinct conflicting category, hybrid neuro-symbolic optimization is presented as a convergence pattern that combines learnt components with precise solvers, constraint programming, mixed-integer programming, local search, and metaheuristic pipelines. Rather than rigorous bibliometric values, branches show methodological progress and convergence.

Five major research gaps that need methodical attention are identified by the critical analysis of the literature studied:

- Distribution-shift robust learning solvers: When test examples deviate from the training distribution, existing trained solutions tend to perform worse. Future techniques should preserve the quality of the solution under various operational settings, structural characteristics, and undiscovered instance sizes.
- Trustworthy neural CO: Formal feasibility and superiority assurances are absent from the majority of neural CO techniques.
- Few-shot and cross-family adaptation: Existing meta-learning work, including [37], mainly targets transfer across related instance distributions. More attention is needed to adaptation across distinct CO problem families, such as routing, scheduling, packing, covering, and graph optimization. We must go beyond simple "black-box" ML and integrate it with symbolic, rule-centered, or formal verification methods that rigorously ensure safety and compliance with constraints in order to trust predictions powered by AI in crucial applications.
- LLM agent-specific standards: Replicability-aware and contamination-assisted benchmarks that include model variations, prompts, deductive estimates, feasibility evaluations, and cost-normalized comparisons with neural, traditional, and metaheuristic baselines are necessary for LLM-driven heuristic synthesis.
- Honest negative outcomes: In the vein of study [59], the field requires further research that clearly investigates whether machine learning enhances performance for a particular CO issue family, including open reporting of neutral or adverse outcomes.

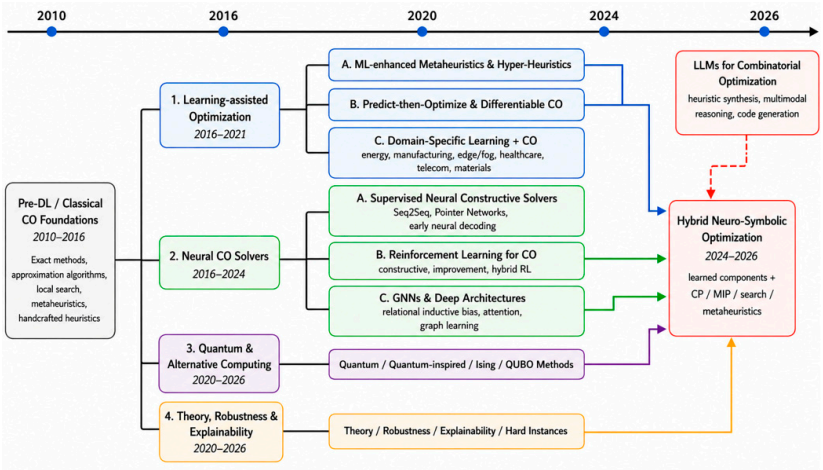


Figure 5. Citation-supported evolutionary taxonomy of ML methods for CO, 2010–2026. The taxonomy encompasses learning-assisted optimization (ML-enhanced metaheuristics and hyper-

heuristics [1,3,5,55–62]; predict-then-optimize and differentiable optimization [12,63–73]; domain-specific learning and optimization [6,18,22–25,38,74–80]), neural combinatorial optimization solvers (supervised neural constructive methods [2,33,73]; reinforcement learning approaches [4,28–37]; GNN and deep architectures [2,11,39–43]), quantum and alternative computing approaches [14,44–54], theory, robustness, and explainability studies [16,81–90], emerging LLM-based optimization methods [7,8,26,27], and hybrid neuro-symbolic optimization frameworks [11,13,31,61].

5. Discussion

This section outlines the evaluated literature throughout ML-for-CO approaches, along with methodological roles and compromises, benchmarking and reproducibility procedures, a detailed discussion of LLM-based optimization, cross-cutting limits, and future research directions.

5.1. Synthesis of ML-for-CO Paradigms: From Standalone Learning to Hybrid Optimization

According to the surveyed literature, hybrid optimization methods that include learnt components in traditional optimization pipelines have replaced the earlier focus on standalone neural solvers in ML-for-CO. While contemporary research increasingly incorporates learning with constraint programming, mathematical programming, local search, and metaheuristics, earlier work mostly focused on supervised and reinforcement learning [1,2,16]. This shift represents a key result of the review: learning is used more to guide, speed, configure, or repair optimization operations than to replace classical optimization in ML-for-CO [1,5].

Reinforcement learning has gained central importance for its ability to work without labeled ideal solutions and to represent combinatorial problems as sequential decision processes [4]. However, training process instability, sensitivity to the objective reward design, sample inefficiency, and limited generalization across different sizes and different distributed cases continue to be limitations of RL approaches [28–31,34]. These drawbacks help to explain why RL is increasingly being used as a component of hybrid systems rather than as a standalone solver.

GNNs provide a complementary contribution by encoding relational dependencies, variable-constraint structure, and permutation-invariant graph representations. They often improve structural generalization relative to earlier sequence-based models [11,40], yet still face over-smoothing, computational cost, and the challenge of mapping continuous embeddings into feasible discrete decisions [39,42,43]. Accordingly, GNNs are increasingly used as representation modules within broader solver-guided pipelines.

ML-enhanced metaheuristics remain among the most practically deployable approaches because they preserve the interpretability and modularity of classical search while adding learned components for operator selection, repair, neighborhood guidance, and search adaptation [55–57,61]. Decision-focused and predict-then-optimize frameworks address a different role by aligning learning with downstream optimization quality rather than prediction accuracy alone [12,63], although they remain constrained by oracle cost, data generation difficulty, and limited evidence under distribution shift [64].

LLMs and quantum/Ising techniques have different but developing roles in this synthesis. Heuristic synthesis, program creation, solver setup, and reasoning assistance are all provided by LLMs; however, external verification and cost-conscious evaluation are necessary [7,8,26,27]. Although alternate QUBO/Ising formulations and specialized computing substrates are offered by quantum and Ising-based techniques [45,49], implementation is still limited by mapping overhead, hardware limitations, and reproducibility issues. When combined, these patterns support hybrid neuro-symbolic optimization as the main guiding concept of current ML-for-CO studies.

A conceptual synthesis of the reviewed literature is presented in Figure 6, which illustrates the evolution of ML-for-CO from classical optimization foundations toward learning-assisted optimization, neural CO, quantum and alternative computing, and theoretical-/robustness-oriented approaches. The figure also highlights the convergence of these branches into hybrid neuro-symbolic frameworks and identifies LLM-based optimization as an emerging direction centered on heuristic synthesis, code generation, and multimodal reasoning.

5.2. Descriptive Trends in the Reviewed Literature

The reviewed literature shows a strong emphasis on routing, production scheduling/planning, graph optimization, and resource allocation, confirming their importance as recurring testbeds for ML-enhanced optimization. It also shows growing methodological diversity: early RL and ML-enhanced approaches are increasingly complemented by graph-based learning, decision-focused optimization, LLM-based optimization, and hybrid learning-optimization frameworks. This development suggests that modern progress is characterized by the coexistence of complementary paradigms rather than by a single dominant methodology.

From an evaluation perspective, synthetic routing, graph optimization, scheduling, and QUBO-based benchmarks remain among the most common experimental settings. Domain-specific datasets in healthcare, manufacturing, communications, cloud computing, and materials science are growing, but their use in systematic comparative evaluation remains limited. Quantitative comparison across methodological families is still constrained by differences in benchmark distributions, evaluation metrics, computational budgets, and baseline solvers. Nevertheless, the most consistent trend across the reviewed studies is the movement toward hybrid approaches that combine learning-based components with traditional search, constraint reasoning, or mathematical optimization.

5.3. Cross-Method Comparative Analysis

A comparative evaluation of the examined approaches demonstrates that the main paradigms in ML-for-CO serve complementary rather than truly competitive roles. RL and GNNs are primarily learning policies and representations, respectively, while ML-enriched metaheuristics and predict-then-optimize techniques incorporate learning processes into current optimization workflows to enhance search performance and decision accuracy. LLM-driven approaches focus on heuristic synthesis, code generation, solver configuration, and reasoning-driven assistance, whereas quantum and Ising-driven methods explore various computational formulations using QUBO/Ising representations and specific hardware. Regardless of these methodological variations, scalability, feasibility enforcement, interpretability, computing overhead, robustness, and deployment feasibility are recurrent trade-offs that all paradigms must contend with. Table 11 outlines the key features, methodological advantages, limitations, and deployment feasibility of the primary ML-for-CO models covered in this review.

Table 11. Methodological comparison of major ML-for-CO models.

Methodology	Primary Role of ML	Main Methodological Advantage	Main Limitation	Practical Deployment Status
Reinforcement learning (RL)	Learn sequential optimization policies	Does not require optimal training labels	Sample inefficiency and reward sensitivity	Practically viable in specific settings
Graph neural Networks (GNNs)	Learn graph-structured representations	Permutation equivariance and structural generalization	Feasibility projection and over-smoothing	Under active methodological development
ML-enhanced Metaheuristics	Improve classical optimization search	Modular, interpretable, and deployment-friendly	Often problem-specific and weakly transferable	Methodologically mature and deployable

Table 11. *Cont.*

Methodology	Primary Role of ML	Main Methodological Advantage	Main Limitation	Practical Deployment Status
Predict-then-optimize	Align learning with downstream decision quality	Strong theoretical grounding	Expensive optimization oracle calls	Practically viable in specific settings
Large language models (LLMs)	Generate heuristics, code, and reasoning strategies	Flexible zero-/few-shot adaptation	Verification, reproducibility, and inference cost	Primarily exploratory in practice
Quantum/Ising Methods	Solve CO through QUBO/Ising formulations	Unified formulation and hardware acceleration potential	Mapping overhead and limited reproducibility	Restricted to specialized experimental settings

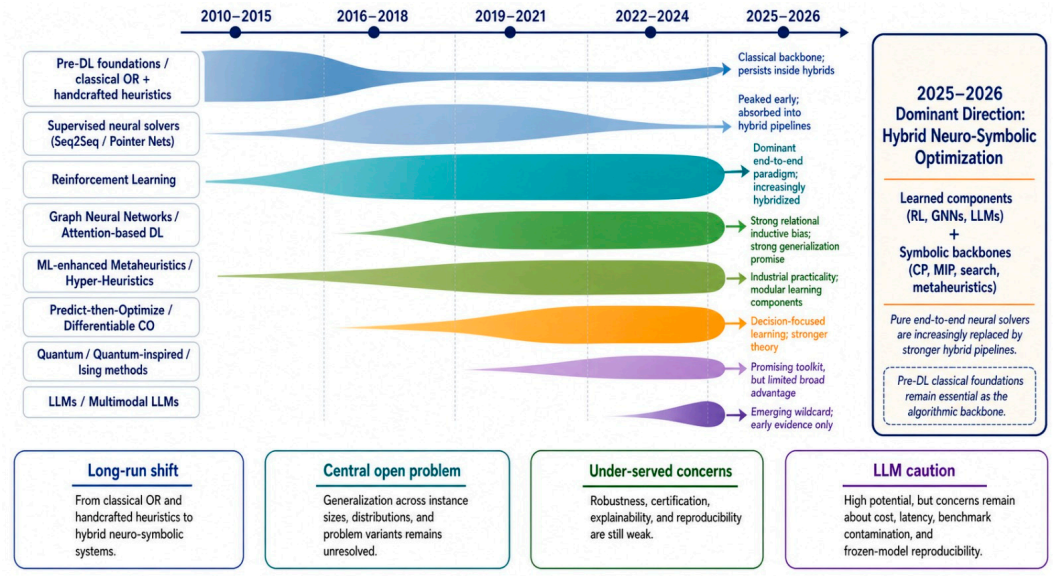


Figure 6. Evolution of ML-for-CO themes from 2010 to 2026.

Table 12 extends the comparison by assessing the reviewed ML-for-CO paradigms across a set of evaluation aspects, including benchmark types, problem scale, statistical validation practices, and reproducibility characteristics. These features provide a deeper comprehension of the robustness, reproducibility, and evaluation process employed by various methodological families.

Table 12. Comparative evaluation characteristics across ML-for-CO methodological families.

Method Family	Benchmark Types	Problem Scale	Statistical Validation Practice	Reproducibility Characteristics
RL	Routing, Scheduling	Small–large synthetic instances	Limited significance testing	Seed- and training-sensitive
GNNs	Graph, Routing	Small–medium training; larger inference	Ablation, cross-instance evaluation	Architecture- and data-dependent
ML-Enhanced Metaheuristics	Routing, Industrial	Medium–large instances	Repeated runs, objective statistics	Generally high
Predict-then-Optimize	Resource, Energy	Problem-dependent	Regret and decision quality metrics	Oracle-dependent
LLM-Assisted Optimization	Routing, Heuristics	Small–medium experimental studies	Limited statistical validation	Prompt- and model-sensitive
Quantum/Ising Methods	QUBO, Max-Cut	Mostly small–medium benchmark instances	Limited cross-platform consistency	Hardware-dependent

5.4. Evaluation, Benchmarking, and Reproducibility Challenges

A persistent challenge across machine learning ML-for-CO is the absence of standardized evaluation methodologies. Direct comparison of the reported results remains difficult because of differences in benchmark selection, evaluation metrics, generalization protocols, computational budgets, and reporting practices employed across studies [1,2,4,16]. Thus, reported improvements should be interpreted with caution because some improvements may be due to choices in experimental design rather than algorithmic supremacy.

Benchmark selection remains a significant source of variability. Standard repositories such as TSPLIB for traveling salesman problems, CVRPLIB for vehicle routing problems, OR-Library benchmark instances, and MIPLIB for mixed-integer optimization are widely adopted, but their instance distributions and structural properties differ significantly [2,16]. Synthetic benchmarks offer several practical advantages, including scalability, controllable difficulty, and reproducible experimentation. Nevertheless, they often fail to capture the real-world constraints, uncertainty, dynamic environments, and structural properties existing in specific domains. Unlike synthetic benchmarks, studies in healthcare, manufacturing, transportation, cloud computing, telecommunications, energy systems, and materials science frequently account for domain-specific requirements, enabling evaluation under conditions that more closely reflect practical deployment scenarios. However, they are less standardized and difficult to compare across domains [6,18,28–31,38,74–80].

The diversity of evaluation measures makes comparison of various approaches more difficult. In the studies of RL, the cumulative reward, regret, or policy quality compared to baseline strategies [4,28–37] are often reported, while the graph-based approaches often report the approximation quality, optimality gap, feasibility rate, or objective function improvement [11,39–43]. In metaheuristic and hyper-heuristic research, evaluation is mostly based on best objective value, convergence behavior, runtime efficiency, and search effectiveness measures [1,3,5,55–62]. Decision-focused learning methods often assess decision loss, regret, or downstream optimization quality [12,63–70], while LLM-based optimization provides additional measures related to heuristic quality, reasoning effectiveness, code generation capability, and solution feasibility [7,8,26,27]. This variation prevents straightforward cross-method comparisons.

Generalization evaluation remains a relatively under-explored aspect of the literature. Current studies adopt various evaluation protocols such as size generalization, distribution generalization, and cross-domain transfer [2,4,11,37,63]. But much of the literature emphasizes distribution evaluation scenarios, where training and testing instances are created from nearly highly similar distributions. Consequently, limited evidence is available regarding the robustness of learned optimization systems when exposed to realistic distribution shifts, evolving constraints, or changing operational environments.

Reproducibility and reporting practices represent additional challenges. Numerous research studies provide insufficient information about random seeds, statistical significance, solver time limits, hardware configurations, memory needs, code availability, and variability over multiple runs. Quantum and specific hardware investigations are limited by platform availability [14,44–54], whereas LLM-based optimization raises problems of prompt sensitivity, closed-model variation, tool chain transparency, and benchmark contamination [7,8,26,27]. Table 13 illustrates the major assessment and reporting problems.

Table 13. Evaluation methodologies and benchmarking practices in ML-for-CO.

Evaluation Aspect	Typical Practice	Main Concern
Benchmarking	Synthetic-dominated evaluation	Weak real-world transferability
Metrics	Paradigm-specific performance measures	Limited comparability
Generalization	Restricted robustness testing	Uncertain out-of-distribution performance
Statistical Validation	Sparse significance analysis	Reliability concerns
Reproducibility	Partial experimental disclosure	Replication barriers
Reporting Practices	Inconsistent efficiency reporting	Difficult practical assessment
LLM/Quantum Evaluation	Prompt- or hardware-dependent evaluation	Validation and accessibility challenges

5.5. LLM-Based Optimization: Opportunities and Reliability Challenges

Because LLM-based optimization modifies the function of learning in CO, it merits further elaboration. LLMs function primarily via language-dependent logic, program synthesis, and tool usage, in contrast to RL and GNN approaches, which often learn representations or policies from problem instances. Heuristic creation is their greatest direct contribution; an LLM can suggest repair methods, constructive rules, local search operators, or solver-guiding code that is then assessed externally. This establishes a neuro-symbolic loop in which traditional solvers, simulators, or benchmark evaluators offer feedback on performance and viability, while the LLM provides potential algorithmic structure.

The second promising use is program synthesis for optimization, where LLMs produce executable routines, modeling code, or solver wrappers that help speed up the creation of problem-specific heuristics. A third approach is tool-augmented optimization agents, where the LLM continually improves solutions by interacting with optimization libraries, MIP/CP solvers, search processes, or verification modules.

But unlike traditional ML-for-CO, this paradigm also creates dependability problems. Feasibility and optimality certificates are rarely provided by LLM-produced solutions, and derived code may include fragile assumptions, concealed implementation defects, or impractical repair logic. Consequently, rather than the fixed model being the final optimization outcome, LLM outputs should be viewed as candidate artifacts that need external confirmation. Another significant issue is benchmark contamination, which undermines claims of generalization by allowing standard CO instances, solver examples, and heuristic templates to appear in pretraining data. Additionally, when prompts, decoding settings, and tool calling procedures are not adequately provided, or when closed-source models change between versions, reproducibility becomes challenging. Lastly, it is necessary to explicitly assess latency and cost. For large batches of CO instances, in particular, repeated LLM calls may be significantly more costly than using customized classical heuristics.

For these causes, LLM-based optimization should be tested with fixed-model procedures, contamination-resistant benchmarks, clear prompt and tool use reporting, feasibility verification, and cost-normalized comparisons to classical, neural, and metaheuristic baselines. Table 14 outlines the functional roles, verification processes, and unresolved hazards associated with LLM-based optimization for CO.

Table 14. Functional roles and unresolved risks of LLM-based optimization for CO.

LLM Role	Function in CO	Verification Mechanism	Main Risk
Heuristic generation	Produces constructive or improvement rules.	Benchmark evaluation/solver comparison.	Prompt sensitivity
Program synthesis	Generates solver code or heuristic routines.	Unit tests/feasibility checks.	Code errors
Tool-augmented agent	Calls solvers, evaluates feedback, revises methods.	External solver validation.	Non-reproducible tool chains
Multimodal reasoning	Uses maps, layouts, or spatial context.	Classical VRP/scheduling baselines.	Small-instance evidence
Solver configuration	Suggests parameters or search strategies.	Runtime and solution quality tests.	Weak generalization
Explanation support	Describes decisions or constraints.	Human/solver consistency checks.	Hallucinated rationale

5.6. Cross-Cutting Limitations and Deployment Barriers

Despite substantial advances in the scope of the review study, several cross-cutting constraints are exposed by combining the preceding sections. Table 15 compares the key ML-for-CO models with respect to scalability, robustness, feasibility, and computational overhead.

Table 15. Methodological limitations of ML-for-CO approaches.

Methodology	Scalability	Robustness	Feasibility	Computational Overhead
Reinforcement Learning (RL)	Scales moderately under controlled instance distributions	Sensitive to distribution shift and reward design instability	Limited without hybrid constraint integration	High training and exploration costs
Graph Neural Networks (GNNs)	Structurally scalable for sparse and moderately sized graphs	Moderate structural transfer across related graph instances	Dependent on projection, repair, or decoding mechanisms	Moderate-to-high representation and inference cost
ML-enhanced Metaheuristics	Highly scalable through preservation of classical optimization backbones	Moderately robust within problem-specific search spaces	Strong due to explicit constraint-aware search procedures	Moderate computational overhead relative to end-to-end neural solvers
Predict-then-Optimize	Constrained by repeated optimization oracle evaluations	Limited evidence under significant distribution shift	Strong through optimization-aware learning formulations	High oracle and differentiation overhead
Large Language Models (LLMs)	Limited by inference latency and large-model computational requirements	Uncertain due to prompt sensitivity and benchmark contamination risk	Lacks formal feasibility and optimality guarantees	Very high inference and verification cost
Quantum/Ising Methods	Restricted by current hardware scalability and embedding constraints	Insufficiently validated across diverse CO settings	Problem-dependent and formulation-sensitive	Specialized hardware and QUBO mapping overhead

The comparative findings reported in Table 15 assure five cross-cutting observations as follows:

1. Hybrid neuro-symbolic pipelines that integrate learnt components (GNN representations, RL policies, LLMs) with symbolic optimization backbones (CP, MIP, metaheuristics) become the most obvious design pattern [7,11,13,31,61].
2. A major open challenge is generalization over instance sizes, problem alternatives, and distributions, despite advances in heterophily-aware architectures, unsupervised relaxations, and meta-learning [37,39,42].
3. Robustness, certification, and explainability are still underdeveloped, specifically for safety-critical or regulated sectors, such as healthcare and energy grids [83,85,88].
4. Quantum and specialized hardware remain intriguing but are limited by embedding, mapping, hardware access, and reproducibility restrictions [49].
5. LLM-based reliability difficulties (discussed in Section 5.5) highlight the general necessity of cost-normalized review, transparent reporting, and verification.

Overall, the ongoing existence of scalability, feasibility, robustness, and deployment issues across all evaluated approaches indicates that no standalone learning architecture presently yields a consistently reliable optimization mechanism. As a result, hybrid neuro-symbolic optimization systems appear as a viable approach for reconciling learning adaptability with formal optimization guarantees.

5.7. Future Research Directions

Based on the cross-cutting limitations identified in the previous section, several important research directions are revealed.

First, future research should implement generalization-aware optimization systems that transfer effectively across graph representations, varying instance sizes, constraint distributions, and problem families. RL, GNN and LLM-based approaches remain vulnerable to performance degradation under distribution shift, motivating frequent employment of meta-learning, transfer learning, uncertainty-aware optimization, and foundation-model-based adaptation.

Second, future work should prioritize certified hybrid neuro-symbolic optimization systems. Learnt components should be combined with exact solver mechanisms such as constraint programming, mixed-integer programming, local search, and verification procedures to ensure feasibility and, when possible, optimality guarantees required for deployment in safety-critical environments.

Third, scalability and computational efficiency continue to represent major open challenges across multiple paradigms. Accordingly, future research should focus on lightweight neural architectures, scalable graph learning mechanisms, adaptive solver selection strategies, sparse and hierarchical optimization representations, and hardware-aware optimization methods that effectively balance solution quality and computational efficiency. This is especially critical because RL training, large-scale GNN message transmission, and LLM inference can significantly increase the computational cost.

Fourth, the development of standard benchmarking frameworks and reproducibility-aware protocols is needed to support robust evaluation and equitable comparison across ML-for-CO paradigms. Future research should include benchmark distributions, solver time restrictions, hardware configurations, random seeds, confidence intervals, memory needs, and statistical validation. Comparisons between LLM-assisted techniques and traditional, metaheuristic, and neural baselines require contamination-resistant benchmarks, standardized reporting standards, and cost-normalized evaluation.

Fifth, Future research on LLM-based optimization should focus on thorough, repeatable evaluation methodologies rather than only showcasing heuristic generation on isolated benchmarks. Contamination-resistant benchmark design, fixed-model and fixed-prompt assessment, consistent prompt and decoding parameter reporting, explicit accounting of inference cost, and solver-based feasibility and solution quality verification are among the top priorities. Stronger reliability measures, such as executable code validation, unit testing, solver log reporting, and protections against erroneous algorithmic assumptions or hallucinated restrictions, are also necessary for tool-augmented LLM agents. Furthermore, with equal computing budgets, future research should compare LLM-generated heuristics with tuned metaheuristics, precise solvers, neural solvers, and hybrid neuro-symbolic pipelines in addition to weak baselines.

Finally, an important direction is to bridge theoretical visions and practical implementations so that advances in complexity theory and robustness analysis can be reliably reflected in real-world optimization systems. Further research on reasoning reliability, heuristic verification, reproducibility under changing model versions, and inference cost scalability is necessary for developing paradigms such as LLM-driven optimization. Simul-

taneously, practical solver design and deployment-specific assessments should be more tightly linked to theoretical developments in complexity analysis, robustness guarantees, and explainable optimization. Lastly, these directions indicate that the future of ML-for-CO will probably rely on scalable, robust, interpretable, and practically deployable hybrid optimization systems that integrate neural learning, symbolic reasoning, and exact optimization capabilities rather than independent learning-based solvers.

6. Conclusions

Supervised neural solvers, reinforcement learning, GNNs, learning-enhanced metaheuristics, predict-then-optimize pipelines, quantum-inspired techniques, domain-specific applications, theoretical research, and new LLM-assisted approaches were all critically examined in this review of machine learning for combinatorial optimization. According to the literature, there is a noticeable shift away from purely end-to-end neural solvers and toward hybrid neuro-symbolic systems, in which components supplement or direct traditional optimization techniques like metaheuristics, local search, large-neighborhood search, mixed-integer programming, and constraint programming. Learning solution structures, speeding up search, choosing heuristics, warm-starting solvers, approximating costly components, and enhancing decision-focused optimization under uncertainty are all areas where ML-for-CO exhibits great promise. The evidence is still inconsistent, though. Narrow benchmarks, weak or inconsistent baselines, restricted statistical testing, inadequate reproducibility, poor generalization analysis, scaling obstacles, and weak feasibility or optimality guarantees are examples of common restrictions. Although it is still in its infancy, LLM-based optimization shows promise. Instead of being a standalone optimizer, the available data supports its use as a heuristic generator with modeling assistance, code generation, or solver support components.

Contamination-resistant benchmarks, cost-normalized evaluation, repeatable protocols, strong feasibility enforcement, stronger baselines, and verified learnt optimization are all necessary for future advancements. Trustworthy ML-for-CO will rely more on the integration of learning techniques with classical optimization than on their replacement. This review looks at the results of recent research, highlights the increasing trend toward hybrid learning-driven optimization frameworks, and highlights significant methodological limitations and open problems.

Author Contributions: Conceptualization, M.E.A.I. and A.E.S.A.; methodology, M.E.A.I.; formal analysis, Y.D. and A.E.S.A.; investigation, M.E.A.I., Y.D. and A.E.S.A.; data curation, M.E.A.I., Y.D., and A.E.S.A.; writing—original draft preparation, M.E.A.I., Y.D., and A.E.S.A.; writing—review and editing, M.E.A.I., and A.E.S.A.; visualization, M.E.A.I. and Y.D.; supervision, M.E.A.I. and A.E.S.A.; All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: No new data were created or analyzed in this study. Data sharing is not applicable to this article.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial Intelligence
CO	Combinatorial Optimization
CP	Constraint Programming
CMA-ES	Covariance Matrix Adaptation Evolution Strategy

DNN	Deep Neural Network
DL	Deep Learning
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
ETO	Engineer-to-Order
GNN	Graph Neural Network
GP	Genetic Programming
IGD	Inverted Generational Distance
IP	Integer Programming
LLM	Large Language Model
LNS	Large Neighborhood Search
Max-Cut	Maximum Cut Problem
MAX-SAT	Maximum Satisfiability Problem
MCTS	Monte Carlo Tree Search
ML	Machine Learning
ML-for-CO	Machine Learning for Combinatorial Optimization
MOO	Multi-Objective Optimization
NAS	Neural Architecture Search
NP-hard	Nondeterministic Polynomial-time Hard
OR	Operations Research
QUBO	Quadratic Unconstrained Binary Optimization
RAN	Radio Access Network
ReAIM	ReRAM-based Adaptive Ising Machine
RL	Reinforcement Learning
RMSE	Root Mean Square Error
RNN	Recurrent Neural Network
SCP	Set Covering Problem
SL	Supervised Learning
SLA	Service-Level Agreement
SME	Small- and Medium-Sized Enterprise
SPO	Smart Predict-and-Optimize
TSP	Traveling Salesman Problem
VM	Virtual Machine
VRP	Vehicle Routing Problem
WoS	Web of Science
XAI	Explainable Artificial Intelligence

References

1. Karimi-Mamaghan, M.; Mohammadi, M.; Meyer, P.; Karimi-Mamaghan, A.M.; Talbi, E.-G. Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art. *Eur. J. Oper. Res.* **2022**, *296*, 393–422. [CrossRef]
2. Wang, F.; He, Q.; Li, S. Solving Combinatorial Optimization Problems with Deep Neural Network: A Survey. *Tsinghua Sci. Technol.* **2024**, *29*, 1266–1282. [CrossRef]
3. Sanchez, M.; Cruz-Duarte, J.M.; Carlos Ortiz-Bayliss, J.; Ceballos, H.; Terashima-Marin, H.; Amaya, I. A Systematic Review of Hyper-Heuristics on Combinatorial Optimization Problems. *IEEE Access* **2020**, *8*, 128068–128095. [CrossRef]
4. Mazyavkina, N.; Sviridov, S.; Ivanov, S.; Burnaev, E. Reinforcement learning for combinatorial optimization: A survey. *Comput. Oper. Res.* **2021**, *134*, 105400. [CrossRef]
5. Bolufé-Röhler, A.; Tamayo-Vera, D. Machine Learning for Enhancing Metaheuristics in Global Optimization: A Comprehensive Review. *Mathematics* **2025**, *13*, 2909. [CrossRef]
6. Zhang, C.; Wu, Y.; Ma, Y.; Song, W.; Le, Z.; Cao, Z.; Zhang, J. A review on learning to solve combinatorial optimisation problems in manufacturing. *IET Collab. Intell. Manuf.* **2023**, *5*, e12072. [CrossRef]
7. Çetinkaya, İ.O.; Büyüktahtakın, İ.E.; Shojaee, P.; Reddy, C.K. Discovering heuristics with Large Language Models (LLMs) for mixed-integer programs: Single-machine scheduling. *Comput. Oper. Res.* **2026**, *186*, 107325. [CrossRef]

8. Albalkhi, S.Y.; Alotaibi, D.F.; Dimitriou, T.; Ahmad, I. Route Optimization Reimagined: Multi-Modal Large Language Models for Next-Generation Vehicle Routing. *IEEE Access* **2026**, *14*, 23835–23865. [CrossRef]
9. Korte, B.; Vygen, J. *Combinatorial Optimization: Theory and Algorithms*; Algorithms and Combinatorics; Springer: Berlin/Heidelberg, Germany, 2018; Volume 21. [CrossRef]
10. Garey, M.R.; Johnson, D.S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*, 27th ed.; A Series of Books in the Mathematical Sciences; W. H. Freeman & Co.: New York, NY, USA, 2009.
11. Cantürk, F.; Varol, T.; Aydoğan, R.; Özener, O.Ö. Scalable Primal Heuristics Using Graph Neural Networks for Combinatorial Optimization. *J. Artif. Intell. Res.* **2024**, *80*, 327–376. [CrossRef]
12. Mandi, J.; Demirovic, E.; Stuckey, P.J.; Guns, T. Smart Predict-and-Optimize for Hard Combinatorial Optimization Problems. In *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Inelegances*; Association for the Advancement of Artificial Intelligence: New York, NY, USA, 2020; Volume 34, pp. 1603–1610. [CrossRef]
13. Charytitsch, B.C.B.; Nascimento, M.C.V. An efficient hybridization of Graph Representation Learning and metaheuristics for the Constrained Incremental Graph Drawing Problem. *Eur. J. Oper. Res.* **2026**, *330*, 381–397. [CrossRef]
14. Heng, S.; Kim, D.; Kim, T.; Han, Y. How to Solve Combinatorial Optimization Problems Using Real Quantum Machines: A Recent Survey. *IEEE Access* **2022**, *10*, 120106–120121. [CrossRef]
15. Vazirani, V.V. *Approximation Algorithms*; Springer: Berlin/Heidelberg, Germany, 2003. [CrossRef]
16. Gambella, C.; Ghaddar, B.; Naoum-Sawaya, J. Optimization problems for machine learning: A survey. *Eur. J. Oper. Res.* **2021**, *290*, 807–828. [CrossRef]
17. Di Caro, G.A.; Maniezzo, V.; Montemanni, R.; Salani, M. Machine learning and combinatorial optimization, editorial. *Spectr.* **2021**, *43*, 603–605. [CrossRef]
18. Yang, X.; Wang, Z.; Zhang, H.; Ma, N.; Yang, N.; Liu, H.; Zhang, H.; Yang, L. A Review: Machine Learning for Combinatorial Optimization Problems in Energy Areas. *Algorithms* **2022**, *15*, 205. [CrossRef]
19. Omidvar, M.N.; Li, X.; Yao, X. A Review of Population-Based Metaheuristics for Large-Scale Black-Box Global Optimization-Part I. *IEEE Trans. Evol. Comput.* **2022**, *26*, 802–822. [CrossRef]
20. Świechowski, M.; Godlewski, K.; Sawicki, B.; Mańdziuk, J. Monte Carlo Tree Search: A review of recent modifications and applications. *Artif. Intell. Rev.* **2023**, *56*, 2497–2562. [CrossRef]
21. Molina-Abril, G.; Calvet, L.; Juan, A.A.; Riera, D. Strategic Decision-Making in SMEs: A Review of Heuristics and Machine Learning for Multi-Objective Optimization. *Computation* **2025**, *13*, 173. [CrossRef]
22. Nayeri, Z.M.; Ghafarian, T.; Javadi, B. Application placement in Fog computing with AI approach: Taxonomy and a state of the art. *J. Netw. Comput. Appl.* **2021**, *185*, 103078. [CrossRef]
23. Boubaker, N.E.H.; Zarour, K.; Guermouche, N.; Benmerzoug, D. A Comprehensive Survey on Resource Management for IoT Applications in Edge-Fog-Cloud Environments. *IEEE Access* **2025**, *13*, 111892–111925. [CrossRef]
24. Palk, M.; Voß, S. Graph Combinatorial Optimization Problems for Blockchain Transaction Network Analysis. *Mathematics* **2026**, *14*, 345. [CrossRef]
25. Burggraef, P.; Wagner, J.; Koke, B.; Steinberg, F. Approaches for the Prediction of Lead Times in an Engineer to Order Environment-A Systematic Review. *IEEE Access* **2020**, *8*, 142434–142445. [CrossRef]
26. Chi, M.; Pang, W.; Wu, X.; Zhao, P.; Li, Y.; Wang, T.; Qian, J.; Xiao, Y.; Wang, L.; Zhou, Y. A generalized neural solver based on LLM-guided heuristic evoluation framework for solving diverse variants of vehicle routing problems. *Expert Syst. Appl.* **2026**, *296*, 128876. [CrossRef]
27. Wu, X.; Wang, D.; Wu, C.; Wen, L.; Miao, C.; Xiao, Y.; Zhou, Y. Efficient Heuristics Generation for Solving Combinatorial Optimization Problems Using Large Language Models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*; ACM: Toronto, ON, Canada, 2025; pp. 3228–3239. [CrossRef]
28. Dolatshah, K.; Toroghi Haghighat, A.; Khajehvand, V.; Hosseini Shirvani, M. Sustainable virtual machine placement in heterogeneous cloud data centers: A reinforcement learning-based approach. *Computing* **2026**, *108*, 17. [CrossRef]
29. Ding, X.; Zhang, Y.; Chen, B.; Ying, D.; Zhang, T.; Chen, J.; Zhang, L.; Cerpa, A.; Du, W. Scalable and Efficient Reinforcement Learning for Virtual Machine Rescheduling in Cloud Data Centers. *IEEE Trans. Parallel Distrib. Syst.* **2026**, *37*, 1186–1204. [CrossRef]
30. Zou, R.; Qin, H.; Xiang, Y.; Wu, C. Handling integrated transportation and production scheduling via deep-Q-network-enhanced multi-objective quality–diversity algorithm. *Eng. Optim.* **2026**, 1–39. [CrossRef]
31. Jesus, A.; Corrêa, A.; Vieira, M.; Marques, C.; Silva, C.; Moniz, S. Enhancing multi-agent deep reinforcement learning for flexible job-shop scheduling through constraint programming. *Comput. Oper. Res.* **2026**, *190*, 107428. [CrossRef]
32. Dantas, A.; do Rego, A.F.; Pozo, A. Using deep Q-network for selection hyper-heuristics. In *GECCO '21: Proceedings of the Genetic and Evolutionary Computation Conference Companion*; Association for Computing Machinery: New York, NY, USA, 2021; pp. 1488–1492. [CrossRef]

33. Gu, S.; Yang, Y. A Deep Learning Algorithm for the Max-Cut Problem Based on Pointer Network Structure with Supervised Learning and Reinforcement Learning Strategies. *Mathematics* **2020**, *8*, 298. [CrossRef]
34. Zhao, F.; Gao, J.; Wang, L.; Sang, H. A Tri-Stage Cooperative Optimization Algorithm with Q-Learning Mechanism for the Multiobjective Distributed Flexible Job Shop Scheduling With Worker Factors. *IEEE Trans. Syst. Man Cybern.-Syst.* **2026**, *56*, 1911–1925. [CrossRef]
35. Xu, M.; Mei, Y.; Zhang, F.; Zhang, M. Niching Genetic Programming to Learn Actions for Deep Reinforcement Learning in Dynamic Flexible Scheduling. *IEEE Trans. Evol. Comput.* **2026**, *30*, 61–75. [CrossRef]
36. Han, S.; Zhang, H.; Li, X.; Yu, J.; Liu, Z.; Zhang, T.; Zheng, X.; Nie, W. Joint Resource Allocation for Underwater Acoustic Cooperative Communication Networks: A Hierarchical Combinatorial Bandit Approach. *IEEE Trans. Cogn. Commun. Netw.* **2026**, *12*, 6104–6118. [CrossRef]
37. Ge, F.; Wang, M.; Chen, D.; Shen, L.; Liu, H. Adaptive Geometry Based Meta-Learning for Multi-Objective Combinatorial Optimization Problems. *Intell. Artif.* **2026**, *20*, 53–66. [CrossRef] [PubMed]
38. Ben Said, A.; Mouhoub, M. Machine Learning and Constraint Programming for Efficient Healthcare Scheduling. *Int. J. Softw. Eng. Knowl. Eng.* **2026**, *36*, 1089–1120. [CrossRef]
39. Guo, X.; Zhang, P.; Cai, Q.; Zhang, Y. Learning to solve combinatorial optimization problems with heterophily. *Neural Netw.* **2025**, *189*, 107554. [CrossRef] [PubMed]
40. Garmendia, A.I.; Ceberio, J.; Mendiburu, A. Neural Improvement Heuristics for Graph Combinatorial Optimization Problems. *IEEE Trans. Neural Netw. Learn. Syst.* **2024**, *35*, 18300–18312. [CrossRef] [PubMed]
41. Wang, R.; Hua, Z.; Liu, G.; Zhang, J.; Yan, J.; Qi, F.; Yang, S.; Zhou, J.; Yang, X. A Bi-Level Framework for Learning to Solve Combinatorial Optimization on Graphs. In *Advances in Neural Information Processing Systems 34 (NEURIPS 2021)*; Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., Vaughan, J., Eds.; Curran Associates Inc.: Red Hook, NY, USA, 2021; Volume 34.
42. Bu, F.; Jo, H.; Lee, S.Y.; Ahn, S.; Shin, K. Tackling prevalent conditions in unsupervised combinatorial optimization: Cardinality, minimum, covering, and more. In *ICML'24: Proceedings of the 41st International Conference on Machine Learning*; MLResearch Press: Norfolk, MA, USA, 2024; Volume 235, pp. 4696–4729.
43. Liu, Y.; Zhou, C.; Zhang, P.; Gao, Y.; Li, Z.; Chen, H. Meta-Heuristics Graph Neural Architecture Search for Combinatorial Optimization. *IEEE Trans. Emerg. Top. Comput. Intell.* **2025**. [CrossRef]
44. Slysz, M.; Grodzki, Ł.; Rydlichowski, P.; Siera, D.; Kurowski, K.; Waligóra, G.; Węglarz, J. Solving combinatorial optimization and machine learning problems on hybrid near-term quantum photonic computers. *Future Gener. Comput. Syst.* **2026**, *174*, 107934. [CrossRef]
45. Jiang, J.-R.; Chu, C.-W. Classifying and Benchmarking Quantum Annealing Algorithms Based on Quadratic Unconstrained Binary Optimization for Solving NP-Hard Problems. *IEEE Access* **2023**, *11*, 104165–104178. [CrossRef]
46. Zeng, Q.-G.; Cui, X.-P.; Liu, B.; Wang, Y.; Mosharev, P.; Yung, M.-H. Performance of quantum annealing inspired algorithms for combinatorial optimization problems. *Commun. Phys.* **2024**, *7*, 249. [CrossRef]
47. Shukla, A.; Erementchouk, M.; Mazumder, P. Non-binary dynamical Ising machines for combinatorial optimization. *Phys. Nonlinear Phenom.* **2025**, *481*, 134809. [CrossRef]
48. Cen, Y.; Das, D.; Fong, X. A tree search algorithm towards solving Ising formulated combinatorial optimization problems. *Sci. Rep.* **2022**, *12*, 14755. [CrossRef] [PubMed]
49. Zaman, M.; Tanahashi, K.; Tanaka, S. PyQUBO: Python Library for Mapping Combinatorial Optimization Problems to QUBO Form. *IEEE Trans. Comput.* **2022**, *71*, 838–850. [CrossRef]
50. Truger, F.; Beisel, M.; Barzen, J.; Leymann, F.; Yussupov, V. Selection and Optimization of Hyperparameters in Warm-Started Quantum Optimization for the MaxCut Problem. *Electronics* **2022**, *11*, 1033. [CrossRef]
51. Hao, T.; Huang, X.; Jia, C.; Peng, C. A Quantum-Inspired Tensor Network Algorithm for Constrained Combinatorial Optimization Problems. *Front. Phys.* **2022**, *10*, 906590. [CrossRef]
52. Ahsan Khandoker, S.; Munshad Abedin, J.; Hibat-Allah, M. Supplementing recurrent neural networks with annealing to solve combinatorial optimization problems. *Mach. Learn. Sci. Technol.* **2023**, *4*, 015026. [CrossRef]
53. Chiang, H.-W.; Nien, C.-F.; Cheng, H.-Y.; Huang, K.-P. ReAIM: A ReRAM-based Adaptive Ising Machine for Solving Combinatorial Optimization Problems. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*; IEEE: Piscataway, NJ, USA, 2024; pp. 58–72. [CrossRef]
54. Huang, K.-P.; Nien, C.-F.; Zhang, Y.-T.; Lee, C.-K.; Wang, Y.-C. GPU-based Ising Machine for Solving Combinatorial Optimization Problems with Enhanced Parallel Tempering Techniques. In *2024 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*; IEEE: Piscataway, NJ, USA, 2024; pp. 636–640. [CrossRef]
55. Garcia, J.; Lemus-Romani, J.; Altimiras, F.; Crawford, B.; Soto, R.; Becerra-Rozas, M.; Moraga, P.; Paz Becerra, A.; Pena Fritz, A.; Rubio, J.-M.; et al. A Binary Machine Learning Cuckoo Search Algorithm Improved by a Local Search Operator for the Set-Union Knapsack Problem. *Mathematics* **2021**, *9*, 2611. [CrossRef]

56. Crawford, B.; Caballero, H.; Astorga, G.; Cisternas-Caneo, F.; Becerra-Rozas, M.; Baeza, A.; Bernales, G.; Puga, P.; Giachetti, G.; Soto, R. A Novel Binary Dream Optimization Algorithm with Data-Driven Repair for the Set Covering Problem. *Biomimetics* **2026**, *11*, 197. [CrossRef] [PubMed]
57. Chen, C.; Wu, J.; Chen, J.; Xia, Y.; Precup, R.-E. Learning-Guided Adaptive Search Optimization for the Weighted Independent Set Problem. *Rom. J. Inf. Sci. Technol.* **2026**, *29*, 89–99. [CrossRef]
58. Hunter, K.; Thomson, S.L.; Hart, E. Variable Importance Estimation for High-Dimensional Optimisation. In *Advances in Computational Intelligence Systems*; Hart, E., Horvath, T., Tan, Z., Thomson, S., Eds.; Springer Nature: Cham, Switzerland, 2026; pp. 115–126. [CrossRef]
59. Dell'Amico, M.; Franchini, G.; Magnani, M.; Zanni, L. Can machine learning help in solving the pallet loading optimization problem? *J. Heuristics* **2026**, *32*, 11. [CrossRef]
60. urasević, M.; umić, M.; Gala, F.J.G. Selection of Automatically Designed Heuristics for the Container Relocation Problem. In *Advances in Computational Intelligence Systems*; Hart, E., Horvath, T., Tan, Z., Thomson, S., Eds.; Springer Nature: Cham, Switzerland, 2026; pp. 15–27. [CrossRef]
61. Chen, R.; Liu, D.; Jiang, N.; Gupta, R.; Kilinc, M.; Lodi, A. Learning large neighborhood search for maritime inventory routing optimization. *Int. Trans. Oper. Res.* **2026**. [CrossRef]
62. Xie, J.; Zhan, J.; Zhu, X. From recursion to prediction: Modeling backtracking effort in TSP with machine learning. *PeerJ Comput. Sci.* **2026**, *12*, e3516. [CrossRef]
63. El Balghiti, O.; Elmachtoub, A.N.; Grigas, P.; Tewari, A. Generalization Bounds in the Predict-Then-Optimize Framework. *Math. Oper. Res.* **2023**, *48*, 2043–2065. [CrossRef]
64. Kotary, J.; Fioretto, F.; Van Hentenryck, P. Learning Hard Optimization Problems: A Data Generation Perspective. In *35th Annual Conference on Neural Information Processing Systems (NeurIPS)*; Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., Vaughan, J., Eds.; Curran Associates Inc.: Red Hook, NY, USA, 2021; Volume 34.
65. Vejar, B.; Aglin, G.; Mahmutogullari, A.I.; Nijssen, S.; Schaus, P.; Guns, T. An Efficient Structured Perceptron for NP-Hard Combinatorial Optimization Problems. In *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research, Patr II, CPAIOR 2024*; Lecture Notes in Computer Science; Dilkina, B., Ed.; Springer Nature: Cham, Switzerland, 2024; Volume 14743, pp. 253–262. [CrossRef]
66. Paulus, A.; Rolinek, M.; Musil, V.; Amos, B.; Martius, G. CombOptNet: Fit the Right NP-Hard Problem by Learning Integer Programming Constraints. In *Proceedings of the 38th International Conference on Machine Learning*; Meila, M., Zhang, T., Eds.; MLResearch Press: Norfolk, MA, USA, 2021; Volume 139, pp. 1–11.
67. Shen, Y.; Sun, Y.; Li, X.; Eberhard, A.; Ernst, A. Adaptive solution prediction for combinatorial optimization. *Eur. J. Oper. Res.* **2023**, *309*, 1392–1408. [CrossRef]
68. Li, M.; Kolouri, S.; Mohammadi, J. Learning to Solve Optimization Problems with Hard Linear Constraints. *IEEE Access* **2023**, *11*, 59995–60004. [CrossRef]
69. Prat, E.; Chatzivasileiadis, S. Learning Active Constraints to Efficiently Solve Linear Bilevel Problems: Application to the Generator Strategic Bidding Problem. *IEEE Trans. Power Syst.* **2023**, *38*, 2376–2387. [CrossRef]
70. Kumar, M.; Kolb, S.; Teso, S.; De Raedt, L. Learning MAX-SAT from Contextual Examples for Combinatorial Optimisation. *Proc. AAAI Conf. Artif. Intell.* **2020**, *34*, 4493–4500. [CrossRef]
71. Marino, R. Learning from survey propagation: A neural network for MAX-E-3-SAT. *Mach. Learn.-Sci. Technol.* **2021**, *2*, 035032. [CrossRef]
72. Liu, J.; Gao, F.; Zhang, J. Gumbel-Softmax Optimization: A Simple General Framework for Combinatorial Optimization Problems on Graphs. In *Complex Networks and Their Applications VIII, Volume 1*; Studies in Computational Intelligence; Cherifi, H., Gaito, S., Mendes, J., Moro, E., Rocha, L., Eds.; Springer: Cham, Switzerland, 2020; Volume 881, pp. 879–890. [CrossRef]
73. Gu, S.; Hao, T.; Yao, H. A pointer network based deep learning algorithm for unconstrained binary quadratic programming problem. *Neurocomputing* **2020**, *390*, 1–11. [CrossRef]
74. Lee, S.; Sohn, S.S.; Lee, H.-S.; Kim, D.; Kang, Y. Accelerating High-Entropy Alloy Design via Machine Learning: Predicting Yield Strength from Composition. *Materials* **2026**, *19*, 196. [CrossRef] [PubMed]
75. Ghiara, E.; Wu, Z.; Voulhoux, M.; Mola Bertran, O.; Bertini, V.; Torres, C.; Kethamkuzhi, A.; Telles, G.T.; Fuentes, V.; Pach, E.; et al. High-Throughput Screening of REBCO Superconducting Thin Films Fabricated Via Combinatorial Inkjet Printing and TLAG Process (Adv. Mater. Technol. 9/2026). *Adv. Mater. Technol.* **2026**, *11*, e70944. [CrossRef]
76. Figalli, A.; Qasim, S.R.; Owen, P.; Serra, N. Designing particle physics experiments with artificial intelligence. *Front. Phys.* **2026**, *14*, 1765091. [CrossRef]
77. Furusawa, S.; Dogo, C.; Saito, K.; Seki, Y.; Kikuchi, S.; Tanaka, S. Comparative evaluation of black-box optimization methods for RAN func-tion placement problem. *IEICE Commun. Express* **2026**, *15*, 21–24. [CrossRef]
78. Uddin, A.; Sakr, A.H.; Zhang, N. Multi-Agent Task Prioritization and Offloading in Vehicular Edge Computing Environments. In *2025 IEEE 102nd Vehicular Technology Conference (VTC2025-Fall)*; IEEE: Piscataway, NJ, USA, 2025; pp. 1–6. [CrossRef]

79. Jiang, N.; Yan, S.; Liu, H.; Peng, M. Computation Offloading for Distributed Learning in Vehicular Networks: A Service Scheduling and Resource Allocation Method. *IEEE Trans. Veh. Technol.* **2026**, *75*, 3222–3237. [CrossRef]
80. C-Sánchez, E.; Gomez, J.F. Enhancing hotel profitability: Dynamic pricing with a Sim-Learnheuristic approach. *Int. J. Hosp. Manag.* **2026**, *133*, 104472. [CrossRef]
81. Gamarnik, D.; Jagannath, A.; Wein, A.S. Low-Degree Hardness of Random Optimization Problems. In *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS 2020); Annual IEEE Symposium on Foundations of Computer Science*; IEEE Computer Society: Piscataway, NJ, USA, 2020; pp. 131–140. [CrossRef]
82. Goldenberg, E.; Karthik, C.S. Hardness Amplification of Optimization Problems. In *11th Innovations in Theoretical Computer Science Conference, (ITCS-2020)*; Leibniz International Proceedings in Informatics; Vidick, T., Ed.; Association for Computing Machinery (ACM): New York, NY, USA, 2020; Volume 151. [CrossRef]
83. Goerigk, M.; Maher, S.J. Generating hard instances for robust combinatorial optimization. *Eur. J. Oper. Res.* **2020**, *280*, 34–45. [CrossRef]
84. Liefooghe, A.; Lopez-Ibanez, M. Many-objective (Combinatorial) Optimization is Easy. In *Proceedings of the 2023 Genetic and Evolutionary Computation Conference, (GECCO-2023)*; Paquete, L., Ed.; Association for Computing Machinery (ACM): New York, NY, USA, 2023; pp. 704–712. [CrossRef]
85. Erwig, M.; Kumar, P. Explanations for combinatorial optimization problems. *J. Comput. Lang.* **2024**, *79*, 101272. [CrossRef]
86. Timofieva, N.K. Artificial Intelligence Problems and Combinatorial Optimization. *Cybern. Syst. Anal.* **2023**, *59*, 511–518. [CrossRef]
87. Khadka, K.; Chandrasekaran, J.; Lei, Y.; Kacker, R.N.; Kuhn, D.R. A Combinatorial Approach to Hyperparameter Optimization. In *Proceedings of the CAIN 2024: IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*; IEEE Computer Society: Piscataway, NJ, USA; Association for Computing Machinery: New York, NY, USA, 2024; pp. 140–149. [CrossRef]
88. Shao, Z.; Yang, J.; Shen, C.; Ren, S. Learning for Robust Combinatorial Optimization: Algorithm and Application. In *IEEE Conference on Computer Communications (IEEE INFOCOM-2022)*; IEEE: Piscataway, NJ, USA, 2022; pp. 930–939. [CrossRef]
89. Xu, J.; Yu, L.; Yang, H.; Ji, S.; Wu, P.; Zhang, Y.; Yang, A.; Li, Q.; Li, H.; Zhu, E.; et al. A special machine for solving NP-complete problems. *Fundam. Res.* **2025**, *5*, 1743–1749. [CrossRef] [PubMed]
90. Jena, S.K.; Subramani, K.; Velasquez, A. A Differential Approach for Several NP-hard Optimization Problems. In *Artificial Intelligence and Image Analysis, ISAIM 2024, IWCIA 2024*; Lecture Notes in Computer Science; Barneva, R., Brimkov, V., Gentile, C., Pacchiano, A., Eds.; Springer Nature: Cham, Switzerland, 2024; Volume 14494, pp. 68–80. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.